CCOpt Rapid Adoption Kit (RAK) for Beginners

Product: **Innovus Version 16.20-p002_1**

Lab Instruction Manual

April 2017

Note: RAK Testcase Database can be downloaded from the 'Attachments' section at the bottom of this PDF. This RAK can also be searched on the support portal i.e. https://support.cadence.com, using the ‘Title’ of the RAK.

Copyright © 2017 Cadence Design Systems, Inc. All rights reserved.

Printed in the United States of America.

Cadence Design Systems, Inc. (Cadence), 2655 Seely Ave., San Jose, CA 95134, USA.

Trademarks: Trademarks and service marks of Cadence Design Systems, Inc. (Cadence) contained in this document are attributed to Cadence with the appropriate symbol. For queries regarding Cadence trademarks, contact the corporate legal department at the address shown above or call 1-800-862-4522.

All other trademarks are the property of their respective holders.

Restricted Print Permission: This publication is protected by copyright and any unauthorized use of this publication may violate copyright, trademark, and other laws. Except as specified in this permission statement, this publication may not be copied, reproduced, modified, published, uploaded, posted, transmitted, or distributed in any way, without prior written permission from Cadence. This statement grants you permission to print one (1) hard copy of this publication subject to the following conditions:

The publication may be used solely for personal, informational, and noncommercial purposes;

The publication may not be modified in any way;

Any copy of the publication or portion thereof must include all original copyright, trademark, and other proprietary notices and this permission statement; and

Cadence reserves the right to revoke this authorization at any time, and any such use shall be discontinued immediately upon written notice from Cadence.

Disclaimer: Information in this publication is subject to change without notice and does not represent a commitment on the part of Cadence. The information contained herein is the proprietary and confidential information of Cadence or its licensors, and is supplied subject to, and may be used only by Cadence customers in accordance with, a written agreement between Cadence and its customer.

Except as may be explicitly set forth in such agreement, Cadence does not make, and expressly disclaims, any representations or warranties as to the completeness, accuracy or usefulness of the information contained in this document. Cadence does not warrant that use of such information will not infringe any third party rights, nor does Cadence assume any liability for damages or costs of any kind that may result from use of such information.

Restricted Rights: Use, duplication, or disclosure by the Government is subject to restrictions as set forth in FAR52.227-14 and DFAR252.227-7013 et seq. or its successor.

Table of Contents

CCOpt RAK for Beginners

Module 1: About This Course 4

Lab 1-1 Locating the Cadence Support Solutions.....5

Lab 1-2 Customizing Notification and Search Preferences.....7

Module 2: Introduction and Setup 9

Lab 2-1 Design the Information and Set up Innovus.....10

2-1 Introduction.....10

2-2 Leon Design Information.....10

2-3 Set up the Innovus Implementation System and the Lab Directory.....11

Module 3: Configuring and Running CCOpt-CTS 14

Lab 3-1 Configuring and Running CCOpt-CTS.....15

3-1 Configuring CCOpt-CTS.....15

Module 4: Running and Analyzing the CCOpt Results 33

Lab 4-1 Running CCOpt and Analyzing its results.....34

4-1 Analyzing the Log file.....34

4-2 CCOpt Analysis Commands.....37

4-3 CCOpt Clock Tree Debugger (CTD).....42

References 55

© 2017 Cadence Design Systems, Inc. All rights reserved worldwide.

Module 1: **About This Course**

© 2017 Cadence Design Systems, Inc. All rights reserved worldwide.

Lab 1-1 Locating the Cadence Support Solutions

Objective: Log in to the Cadence® Support website and search for application notes on ccopt_design.

You can complete this lab only if you have access to the Internet and have a COS account. If you do not have a COS account, do register yourself on Cadence Support website

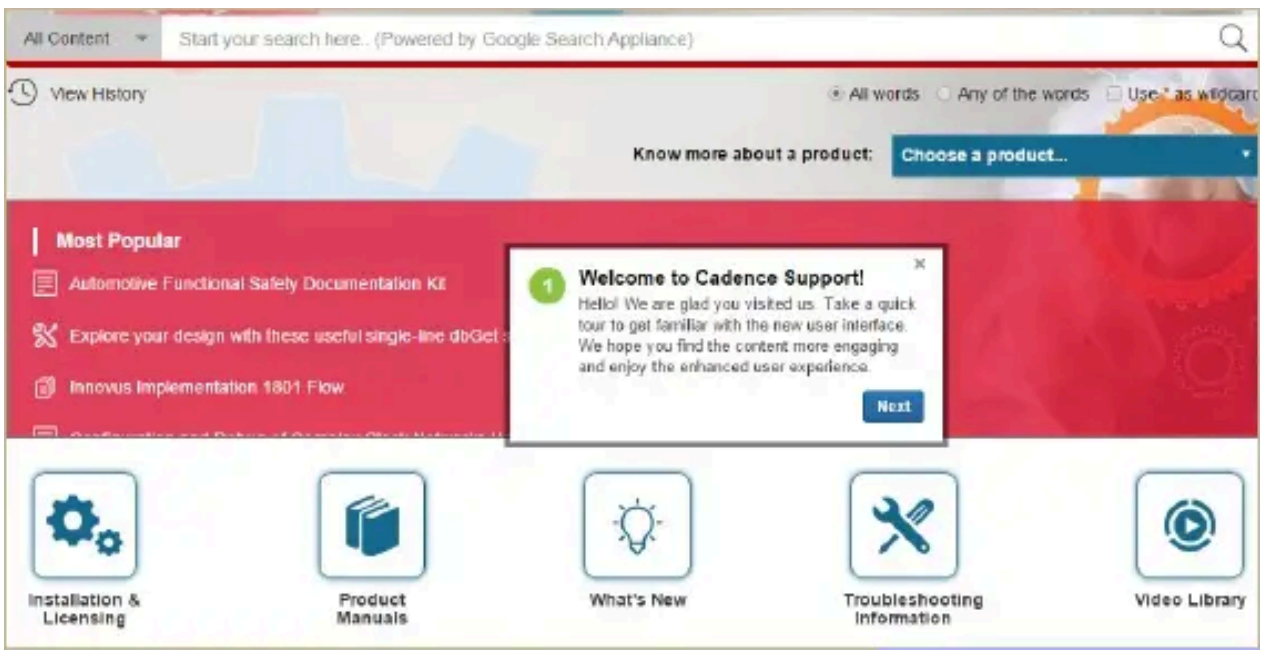
1. In a web browser, enter the following URL:

   https://support.cadence.com

2. Log in to COS with your email ID and password.



The COS Home page appears.



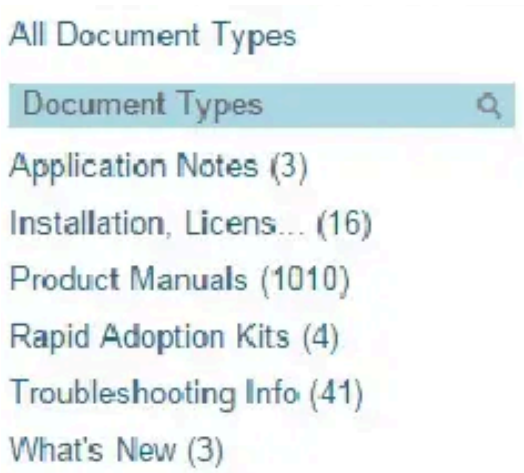
© 2017 Cadence Design Systems, Inc. All rights reserved worldwide.

3. In the Search field, enter the following keyword:

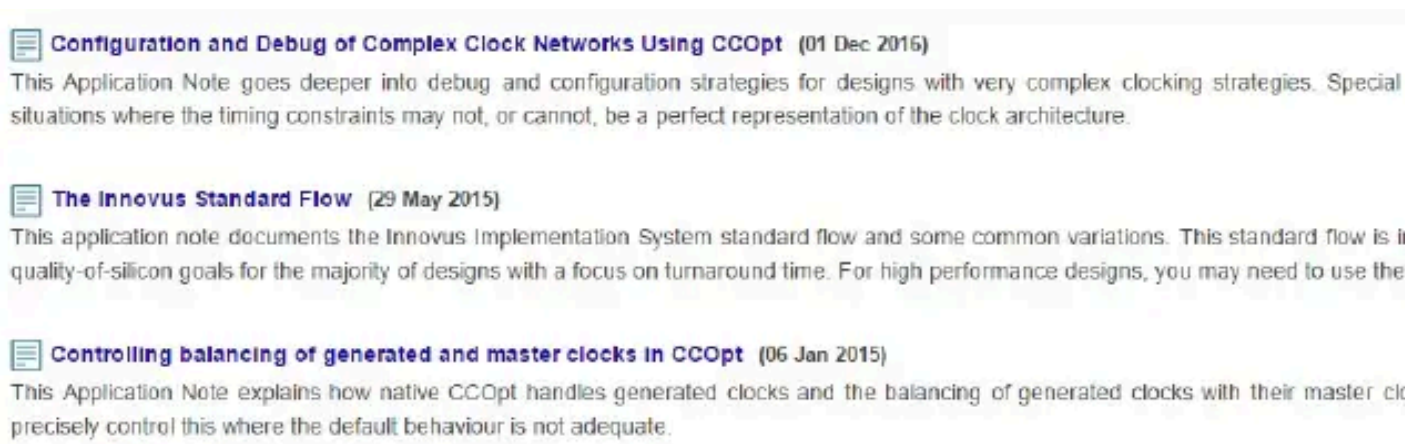
   ```
   ccopt_design
   ```

   A window displays the search results.

4. On the left side, under **Document Types**, click **Application Notes**:



5. In the **Application Notes Info** section, you can view the following notes:



   Click the link you want to view.

6. Close the window.

   Note: You can filter the search results by selecting a specific document type or product that is listed to the left of the results.

© 2017 Cadence Design Systems, Inc. All rights reserved worldwide.

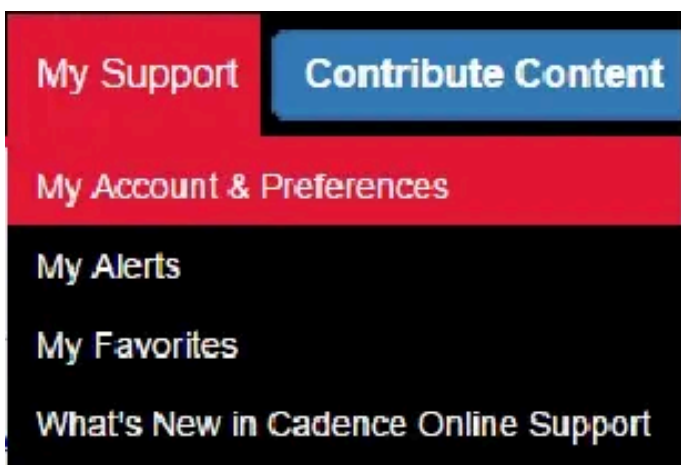
Lab 1-2 Customizing Notification and Search Preferences

Objective: Set the preferences to improve your search results and to receive email notifications.

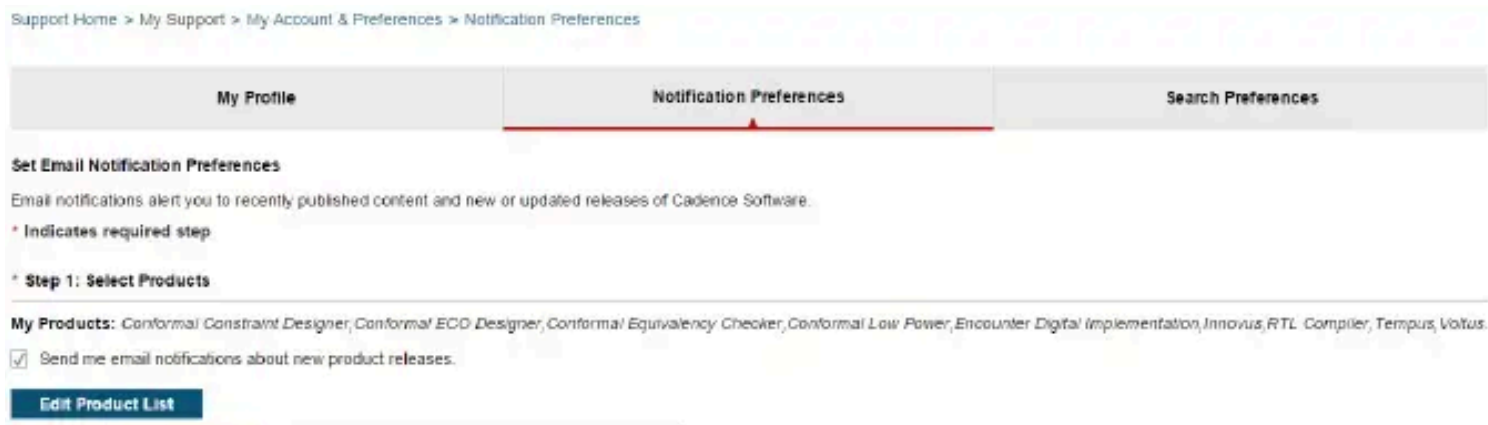
You can set the product and other preferences for improved search results and email notifications.

1. Place your mouse pointer on the **My Support** tab.

2. Click the **My Account & Preferences** menu.



3. Click the **Notification Preferences** tab to open it. Then, perform the following actions:



   a. If you are interested in receiving email notifications, enable the **Send me email notifications about new product releases** check box.

   b. Click **Edit Product List** and select your products of interest.

   c. Specify your preferred email format.

   d. Select the document types you are interested in and the frequency of delivery.

   e. Click **Save**.

© 2017 Cadence Design Systems, Inc. All rights reserved worldwide.

4. Click the **Search Preferences** tab.

5. On the **Set Search Preferences** page, perform the following actions:

   a. Click **Use same product and document type preferences as my Notification Preferences**.

   b. Click **Save**.



© 2017 Cadence Design Systems, Inc. All rights reserved worldwide.

Module 2: **Introduction and Setup**

© 2017 Cadence Design Systems, Inc. All rights reserved worldwide.

Lab 2-1 Design the Information and Set up Innovus

Objective: Check the design and its SDC and run Innovus by restoring the pre-CTS db.

2-1 Introduction

The Clock Concurrent Optimization (CCOpt) Rapid Adoption Kit (RAK) introduces you to running CCOpt-CTS and CCOpt in the Innovus Implementation System. It shows the basics of how to configure and run the CCOpt plus tools and commands to analyze and debug the results. It assumes that you are familiar with running the Innovus Implementation System.

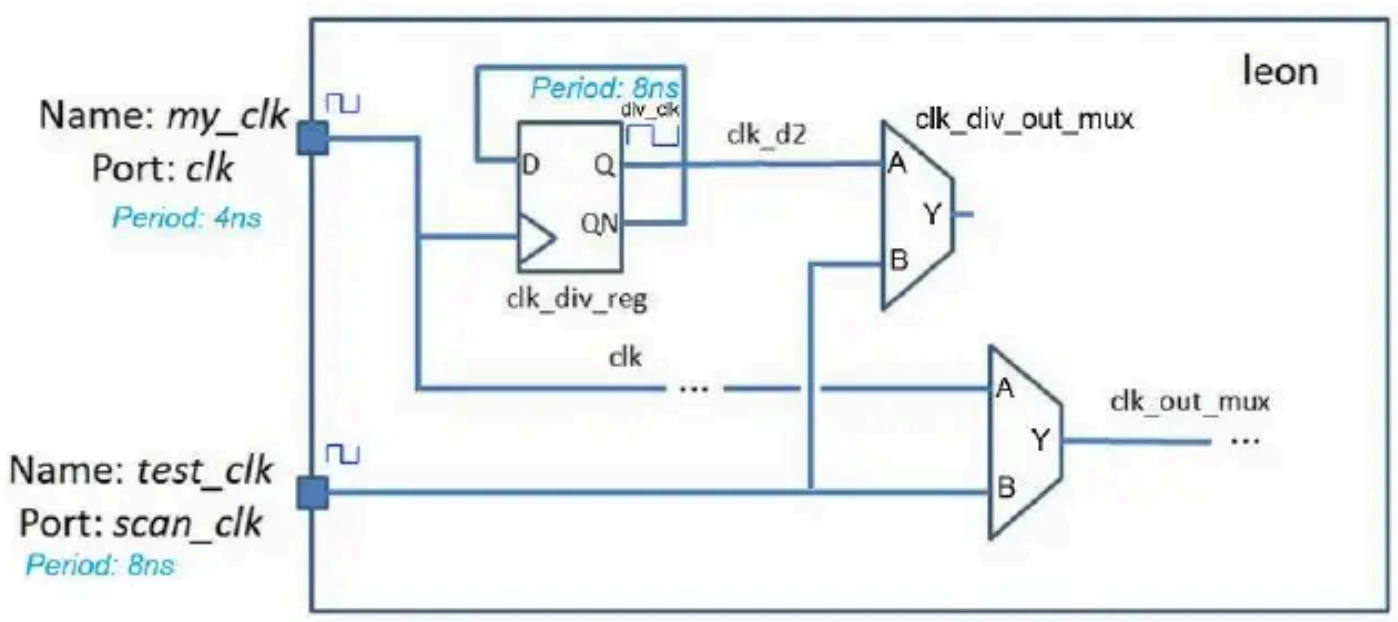
2-2 Leon Design Information

The design in this lab is a Leon processor. The Leon design is a block-level design with 35K instances, 4 memories, and 1211 IO pins. The library used is a Cadence Generic 45nm library using 9 routing layers.

There are three main clocks in this design (*test_clk*, *my_clk*, and *div_clk*), as defined by the following SDCs:

```
create_clock -name {test_clk} -period 8.000 -waveform {0.000 4.000} \
[list [get_ports {scan_clk}]]
create_clock -name {my_clk} -period 4.000 -waveform {0.000 2.000} \
[list [get_ports {clk}]]
create_generated_clock -name div_clk -source [get_ports {clk}] \
-divide_by 2 [get_pins {clk_div_reg/Q}]
```

A schematic of the clock structure is as follows:



© 2017 Cadence Design Systems, Inc. All rights reserved worldwide.

2-3 Set up the Innovus Implementation System and the Lab Directory

1. Download and install the Innovus 16.20 software from https://download.cadence.com.

   a. Extract the RAK database and change the directory to the working directory:

      ```
      linux% tar xfz RAK_16.2_CCOpt.tgz
      linux% cd RAK_16.2_CCOpt
      ```

   b. Verify that the `innovus` executable is in your path by typing the following command:

      ```
      linux% which innovus
      ```

   c. Start the Innovus System:

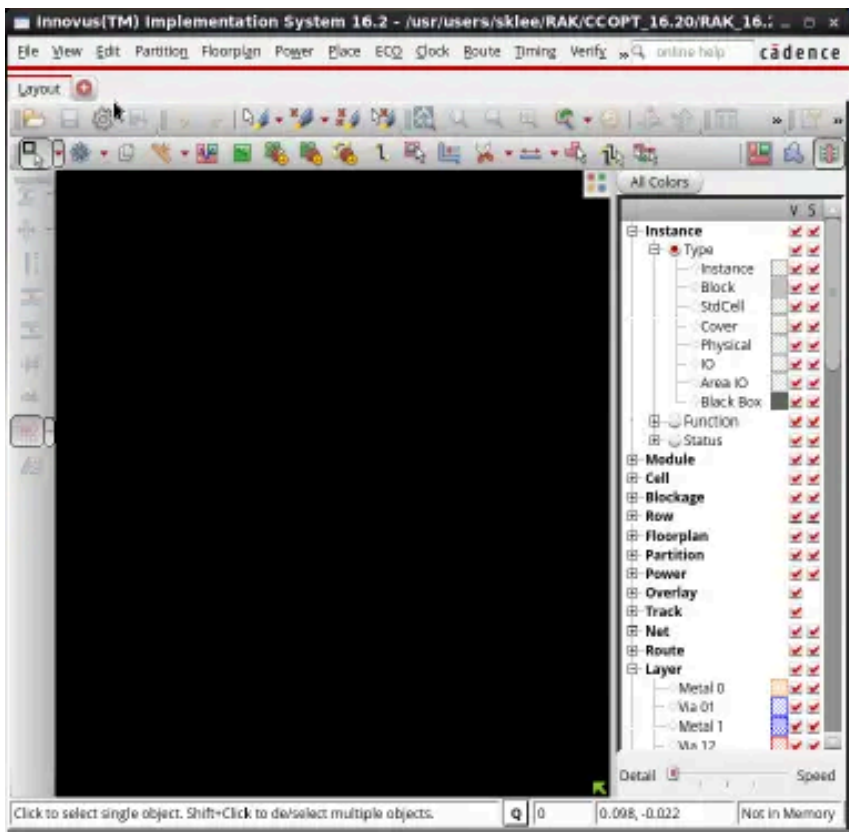
      ```
      linux% innovus
      ```

   d. Load the starting design by running the following command in the Innovus System console:

      ```
      innovus> source DATA/prects.enc
      ```

      Otherwise, you can do as follows on the GUI:

      An empty design window appears as follows:



© 2017 Cadence Design Systems, Inc. All rights reserved worldwide.

On the **File** pull-down menu, choose **Restore Design** to load an Innovus design:



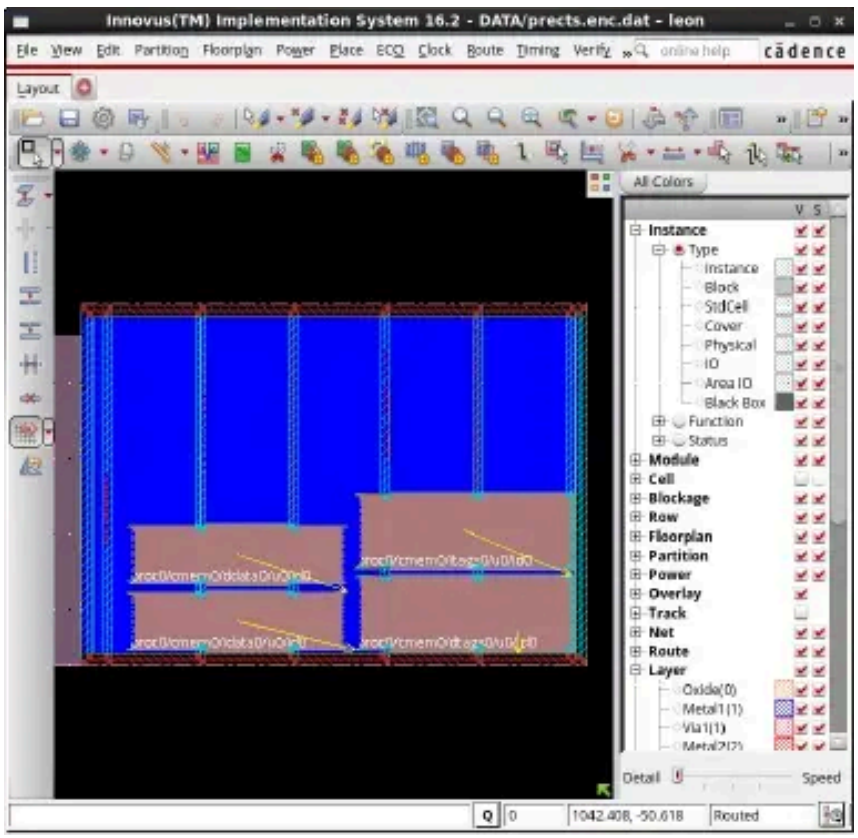
The **Restore Design** window opens.

Click **Folder**, click the **DATA** directory folder, select the *prects.enc* file, and click **OK**.

You see the following window:



Click **OK** and the *DATA/prects.enc* design file is loaded.



© 2017 Cadence Design Systems, Inc. All rights reserved worldwide.

You can check the number of total instances and total memory instances of the *leon* design as follows:

```
innovus> current_design
leon


innovus> sizeof_collection [get_cells -hier -filter
"is_hierarchical == false"]
35773
innovus> sizeof_collection [get_ports *]
1211
innovus> get_cells -hier -filter "is_memory_cell ==
true"
proc0/cmem0/ddata0/u0/id0 proc0/cmem0/dtags0/u0/id0
proc0/cmem0/idata0/u0/id0 proc0/cmem0/itags0/u0/id0
```

Exit the Innovus Implementation as follows:

```
innovus> exit
```



© 2017 Cadence Design Systems, Inc. All rights reserved worldwide.

Module 3: **Configuring and Running CCOpt-CTS**

© 2017 Cadence Design Systems, Inc. All rights reserved worldwide.

Lab 3-1 Configuring and Running CCOpt-CTS

Objective: Configure CCOpt-CTS and run ccopt_design -cts

3-1 Configuring CCOpt-CTS

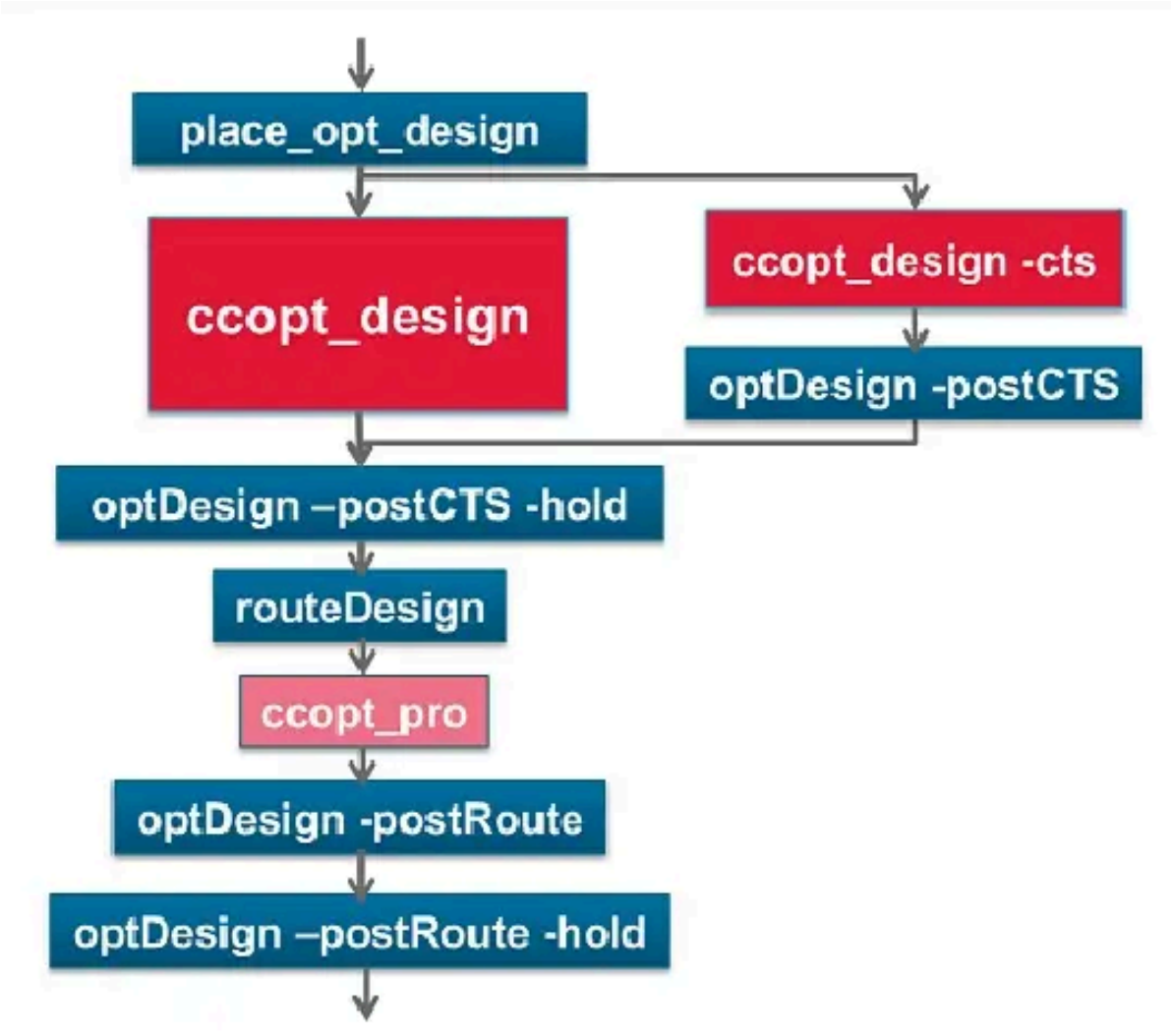
The Innovus Implementation System product family offers two types of clock tree synthesis (CTS) engines:

- CCOpt: Full Clock Concurrent Optimization (CCOpt).

The *ccopt_design* command is used to invoke this.

- CCOpt-CTS: Global skew-balanced CTS using the CCOpt CTS engine.

The *ccopt_design -cts* command is used to invoke this. **CCOpt and CCOpt-CTS in the Design Flow**



© 2017 Cadence Design Systems, Inc. All rights reserved worldwide.

In this module, you will focus on running CCOpt-CTS and in the next module, you will focus on running CCOpt.

Before running CCOpt-CTS and CCOpt, you need to configure a number of settings that control how CCOpt-CTS and CCOpt optimize the design. These settings include:

- Post-CTS timing settings
- Route type settings
- Cell list settings
- CTS constraint settings
- NanoRoute settings

The configuration settings for this lab are contained in the *./SCRIPTS/config.tcl* file.

1. Open the *config.tcl* file by using the vi editor:

```
linux% vi ./SCRIPTS/config.tcl
```

The following describes the settings configured by config.tcl. As CCOpt-CTS and CCOpt are timing-driven, you need to configure CCOpt to use post-CTS timing. That is, you need to configure CCOpt to use the post-CTS SDC files, timing derates, and CPPR settings. This post-CTS timing configuration change needs to be done before running CCOpt.

The relevant commands in *config.tcl* are as follows:

```
### Post-CTS timing setup

update_constraint_mode -name functional_func_slow_max \

-sdc_files [list DATA/leon_func_slow_max_postcts.sdc]

setAnalysisMode -analysisType onChipVariation -cppr both
```

You may need to define the route types to specify the preferred routing layers:

```
### Route type settings

create_route_type -name leaf -top_preferred_layer 5 \

-bottom_preferred_layer 4 –preferred_routing_layer_effort high

create_route_type -name trunk -top_preferred_layer 6 \

-bottom_preferred_layer 5 –preferred_routing_layer_effort high
```


© 2017 Cadence Design Systems, Inc. All rights reserved worldwide.

Specify that these route types will be used for leaf, trunk, and top nets respectively. Note that the top routing rules will not be used unless the routing_top_min_fanout property is also set:

```
set_ccopt_property route_type -net_type top trunk

set_ccopt_property route_type -net_type trunk trunk

set_ccopt_property route_type -net_type leaf leaf
```

There are a number of CCOpt control settings you need to configure to specify the buffers, inverters, and clock gating cells that will be used:

```
### CCOpt control settings

set_ccopt_property buffer_cells \

{CLKBUFX8 CLKBUFX12 CLKBUFX16 CLKBUFX20}

set_ccopt_property inverter_cells \

{CLKINVX8 CLKINVX12 CLKINVX16 CLKINVX20}

set_ccopt_property clock_gating_cells {TLAT*}
```

By default, CCOpt attempts to automatically find the suitable cells. However, it is strongly recommended that you supply a list of the suitable cells. This is because in many libraries, the cells you want to use for CTS are marked as “don’t use”, and CCOpt does not use these cells automatically.

You may also need to set some CTS constraints like skew target and max transition target:

```
set_ccopt_property target_skew 0.200

set_ccopt_property target_max_trans 0.200
```

Tip: Use `get_ccopt_property` to get help on the properties:

- get_ccopt_property * -help – to list all properties
- get_ccopt_property *string* -help – for help on properties matching *string*. For

© 2017 Cadence Design Systems, Inc. All rights reserved worldwide.

example, get_ccopt_property *skew* -help

- get_ccopt_property property_name –help – for help on a specific property

In order to route the clock trees correctly, you need to configure the NanoRoute settings. Of particular interest is the `setNanoRouteMode -drouteUseMultiCutViaEffort [high|low]` command because CCOpt uses this to predict whether NanoRoute will use double vias (high) or single vias (low).

The relevant commands in *config.tcl* are as follows:

```
### NanoRoute settings for clock routing
setNanoRouteMode -drouteUseMultiCutViaEffort "high"
```


© 2017 Cadence Design Systems, Inc. All rights reserved worldwide.

3-2 Creating the CCOpt Spec File

The following command enables you to run CTS in a different mode for debugging purposes:

```
set_ccopt_property balance_mode cluster|trial|full
```

cluster: enables DRV buffering, but does not perform any balancing or optimization of the clock tree.

trial: enables DRV buffering and uses virtual delays to approximate how a full CTS will balance the clock trees.

full: a full CTS or optimization is performed and this is the default type.

You can proceed to run either CCOpt or CCOpt-CTS with the spec file generated in the previous section.

1. Run the `runInit.tcl` script. This creates a clock spec file, reads it into Innovus, and then saves its db:

   ```
   linux% innovus -log ./LOGS/Init.log -init
   ./SCRIPTS/runInit.tcl
   ```

   The tcl file includes:

   ```
   source ./DATA/prects.enc
   source ./SCRIPTS/config.tcl
   create_ccopt_clock_tree_spec -file ccopt.spec
   source ./ccopt.spec
   saveDesign ./DBS/ctsInit.enc
   ```

Invoke the CCOpt clock tree debugger (CTD) to navigate the clock trees.

```
innovus> ctd_win
```


© 2017 Cadence Design Systems, Inc. All rights reserved worldwide.

Observe the following clock trees:



Also, try to find the ignore pin in the *./ccopt.spec* clock spec file:

```
set_ccopt_property sink_type -pin mcore0/a0/g1626/B ignore
set_ccopt_property sink_type_reasons -pin mcore0/a0/g1626/B
no_sdc_clock
```


© 2017 Cadence Design Systems, Inc. All rights reserved worldwide.

Then, try to understand why it is set to an ignore pin by CCOpt.

```
innovus> report_property [get_pins mcore0/a0/g1626/B]
property                                  | value
------------------------------------------------------
…
arrival_max_fall                          | 6.299
arrival_max_rise                          | 2.179
…
is_clock                                  | false
is_clock_gating                           | false
is_clock_gating_clock                     | false
is_clock_gating_enable                    | false
is_clock_used_as_clock                    | false
is_clock_used_as_data                     | true
```

```
innovus> all_fanin -to mcore0/a0/g1626/B
mcore0/a0/g1626/B clk_div_out_mux/Y clk_div_out_mux/S0
clk_div_out_mux/B clk_div_out_mux/A clk_div_reg/Q   tm scan_clk

innovus> report_ccopt_skew_groups -summary
```


© 2017 Cadence Design Systems, Inc. All rights reserved worldwide.

```
 Skew Group Structure:
 =====================
--------------------------------------------------------------------------------
Skew Group                              Sources    Constrained Sinks    Ignore Sinks
--------------------------------------------------------------------------------
div_clk/functional_func_slow_max          1             1922                1
my_clk/functional_func_slow_max           1            11536                1
test_clk/functional_func_slow_max         1            11536                1
--------------------------------------------------------------------------------
```

```
innovus> get_ccopt_property -skew_group my_clk/functional_func_slow_max ignore_pins
innovus> get_ccopt_property sink_type -pin -help

Usage: get_ccopt_property sink_type:
The type of sink this pin represents.

Valid values are as follows:

auto      The pin type will be automatically determined by CCOpt
through   Through pin. Trace the clock tree through this pin.
stop      Stop pin. When defining clock trees, CCOpt stops searching for parts
          of the clock tree at stop pins.
ignore    Ignore pin. CCOpt stops searching for parts of the clock tree at ignore
          pins and it does not attempt to balance the insertion delay of ignore
          pins.
min       Min pin. Keep the pin at minimal insertion delay.
exclude   Exclude pin. Exclude this pin from the clock tree.

Valid values: auto, through, stop, ignore, min, exclude

Default: auto

Applicable arguments: "-pin <name>". Required: "-pin <name>".

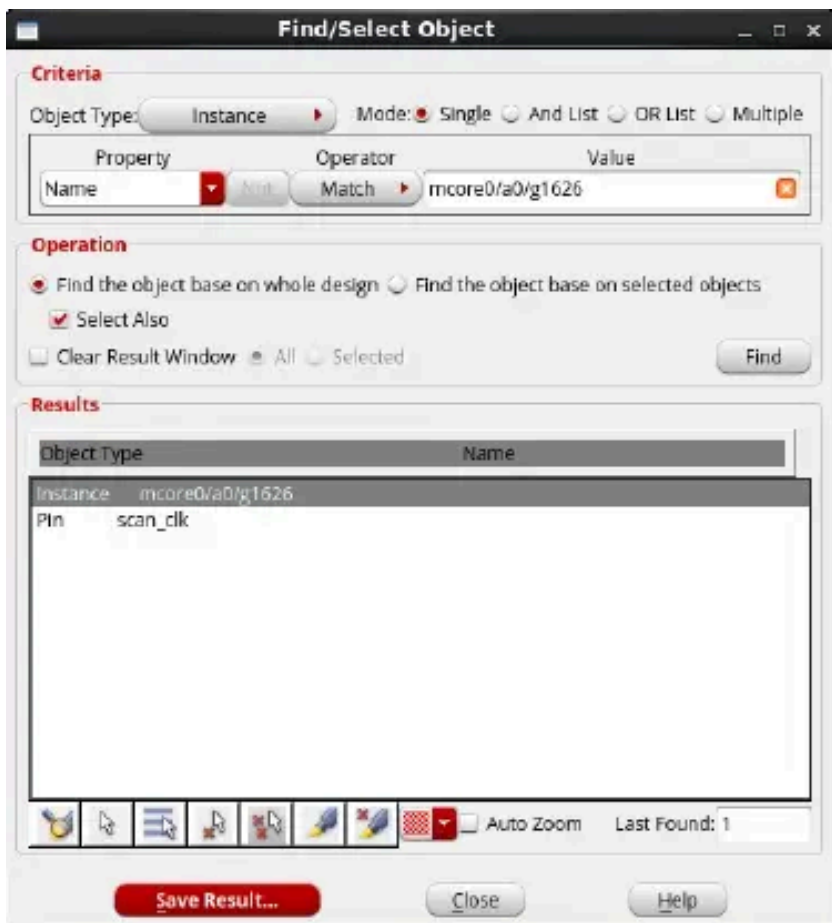
get_ccopt_property -pin:
No matching properties found.

 innovus> get_ccopt_property sink_type -pin mcore0/a0/g1626/B

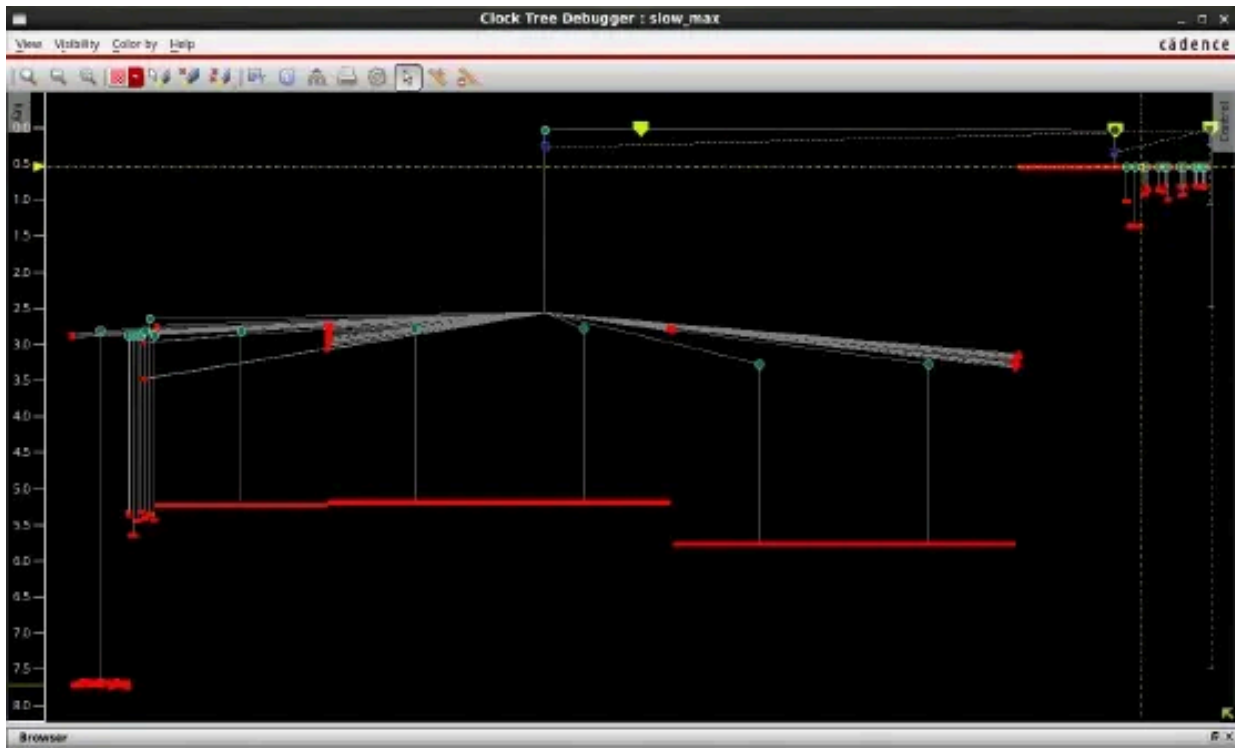
 ignore
```


© 2017 Cadence Design Systems, Inc. All rights reserved worldwide.

Click **Find/Select Object**, enter mcore0/a0/g1626 in the **Value** field, and click **Find**. Then, double-click the **"Instance mcore0/a0/g1626"** item.



You can see where the ignore pin exists as follows:



In these clock trees, you can see 2 MUXs, 36 integrated clock gating cells, and 1 FF in the clock trees.

```
innovus> llength [get_ccopt_clock_tree_cells *]
39
```


© 2017 Cadence Design Systems, Inc. All rights reserved worldwide.

Write a file named `cell_cnt.tcl` as follows:

```
array unset cnt
foreach _cell [get_ccopt_clock_tree_cells *] {
   if {[get_property [get_cells $_cell] object_type] == "cell"} {
     set _ref_lib_cell_name [get_property [get_cells $_cell] ref_lib_cell_name]
     #Puts "$_cell ($_ref_lib_cell_name)"
     if {[info exist cnt($_ref_lib_cell_name)]} {
       incr cnt($_ref_lib_cell_name)
     } else {
       set cnt($_ref_lib_cell_name) 1
     }
   }
 }
foreach key [array names cnt] {
 puts "${key}  : $cnt($key)"
}
```

Then, run the tcl as follows:

```
innovus> source cell_cnt.tcl

TLATNTSCAX2 : 36

DFFX1 : 1

MX2X4 : 2
```

Finally, exit the Innovus session.

3-3. Running CCOpt-CTS with cluster mode

The following command enables you to run CCOpt-CTS with cluster mode:

```
set_ccopt_property balance_mode cluster
```

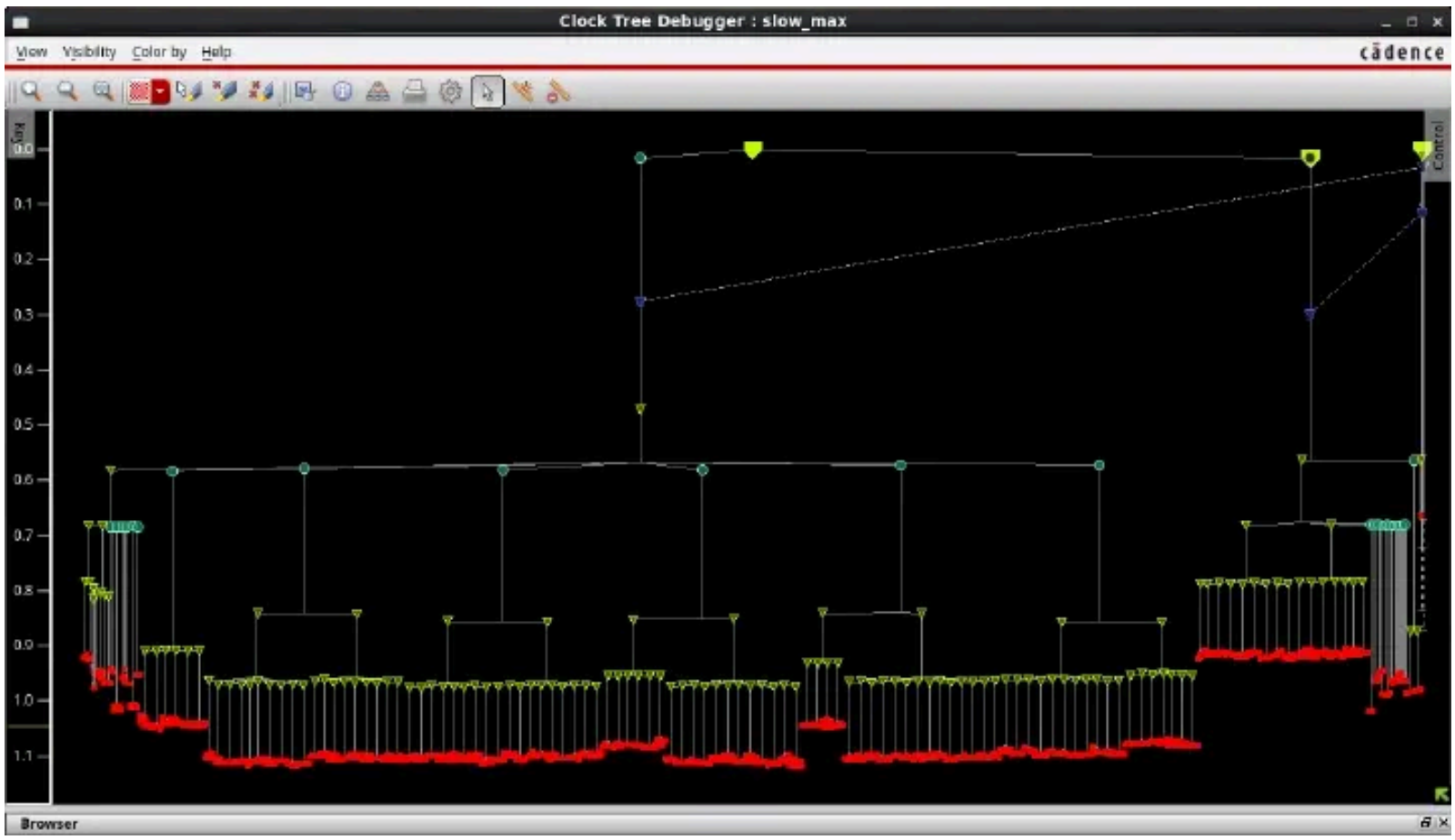
1. Run CCOpt-CTS with cluster to check how the buffers are inserted for DRV fixing:

   ```
   linux% innovus -log ./LOGS/Cluster.log -
   init./SCRIPTS/runCluster.tcl

   innovus> ctd_win
   ```


© 2017 Cadence Design Systems, Inc. All rights reserved worldwide.

Observe that some buffers are inserted into the clock trees.




```
innovus> sizeof_collection [get_cells -hier -filter "name =~
CTS*"]

146

innovus> llength [get_ccopt_clock_tree_cells *]

185
```

In conclusion, 146 cells are inserted through cluster CTS. Before cluster CTS, the number of clock tree cells was 39. After cluster CTS, the number of clock tree cells became 185.

© 2017 Cadence Design Systems, Inc. All rights reserved worldwide.

You can check which type of CTS cells are inserted. There are two kinds of CTS cells named "CTS*ccl*" and "CTS*ccd*".

```
innovus> get_cells -hier -filter "name =~ CTS*"

mcore0/a0/CTS_ccd_BUF_div_clk_G0_L2_2
mcore0/CTS_ccl_BUF_div_clk_G0_L4_10
mcore0/CTS_ccl_BUF_div_clk_G1_L3_2
mcore0/CTS_ccl_BUF_div_clk_G1_L3_1
proc0/cmem0/ddata0/u0/CTS_ccl_BUF_my_clk_G1_L6_101
proc0/cmem0/dtags0/u0/CTS_ccl_BUF_my_clk_G1_L6_100
proc0/cmem0/dtags0/u0/CTS_ccl_BUF_my_clk_G1_L6_94
proc0/cmem0/idata0/u0/CTS_ccl_BUF_my_clk_G1_L6_99 …

innovus> get_cells [get_ccopt_clock_tree_cells *] -filter "name
!~ CTS_ccl* and name =~ CTS*"

mcore0/a0/CTS_ccd_BUF_div_clk_G0_L2_2

   0x2b91

innovus> all_fanout -from
mcore0/a0/CTS_ccd_BUF_div_clk_G0_L2_2/Y -only_cells -pin_levels
2

  mcore0/a0/CTS_ccd_BUF_div_clk_G0_L2_2 mcore0/a0/g1626

   0x2b93
```

You can check that the clock buffers are inserted and the MUX and ICG cells are sized as follows:

```
innovus> source cell_cnt.tcl
TLATNTSCAX2 : 14
TLATNTSCAX20 : 3
TLATNTSCAX3 : 3
CLKBUFX20 : 99
TLATNTSCAX4 : 2
MX2X4 : 1
TLATNTSCAX6 : 8
TLATNTSCAX8 : 3
MX2X8 : 1
DFFX1 : 1
TLATNTSCAX12 : 1
CLKBUFX12 : 6
TLATNTSCAX16 : 2
CLKBUFX16 : 39
CLKBUFX8 : 2
```


© 2017 Cadence Design Systems, Inc. All rights reserved worldwide.

Exit the Innovus session and view the following Runtime Summary after cluster CTS in the *./LOGS/Cluster.log* file:

```
Runtime Summary:
================

-----------------------------------------------------------------------------------------------------
wall    % time  children  called  name
-----------------------------------------------------------------------------------------------------
132.54  100.00   132.54     0
132.54  100.00   131.98     1     Runtime
  0.46    0.35     0.46     1     CCOpt::Phase::Initialization
  0.46    0.35     0.46     1         Check Prerequisites
  0.46    0.35     0.00     1         Leaving CCOpt scope - CheckPlace
  3.39    2.56     0.00     1         Leaving CCOpt scope - optDesignGlobalRouteStep
  5.83    4.40     0.00     1         Validating CTS configuration
  0.59    0.44     0.00     1         Preparing To Balance
102.39   77.26   102.39     1     CCOpt::Phase::Clustering
 40.01   30.19    34.33     1       Stage::DRV Fixing
 29.92   22.58     2.14     1         Clustering
  2.14    1.61     0.00     1         Leaving CCOpt scope - RefinePlacement
  1.27    0.96     0.00     1         Leaving CCOpt scope
  2.00    1.51     0.00     1         Fixing clock tree slew time and max cap violations
  1.15    0.87     0.00     1         Fixing clock tree slew time and max cap violations - detailed pass
 62.38   47.07    62.26     1       Stage::Insertion Delay Reduction
  0.92    0.69     0.00     1         Removing unnecessary root buffering
  0.74    0.56     0.00     1         Removing unconstrained drivers
  0.85    0.64     0.00     1         Reducing insertion delay 1
  2.54    1.91     0.00     1         Removing longest path buffering
 57.21   43.17     0.00     1         Reducing insertion delay 2
  7.66    5.78     1.26     1     CCOpt::Phase::Implementation
  1.26    0.95     0.00     1         Leaving CCOpt scope
 11.65    8.79     0.00     1         Tidy Up And Update Timing
-----------------------------------------------------------------------------------------------------
```

3-4. Running CCOpt-CTS with trial mode

The following command enables you to run CCOpt-CTS with trial mode:

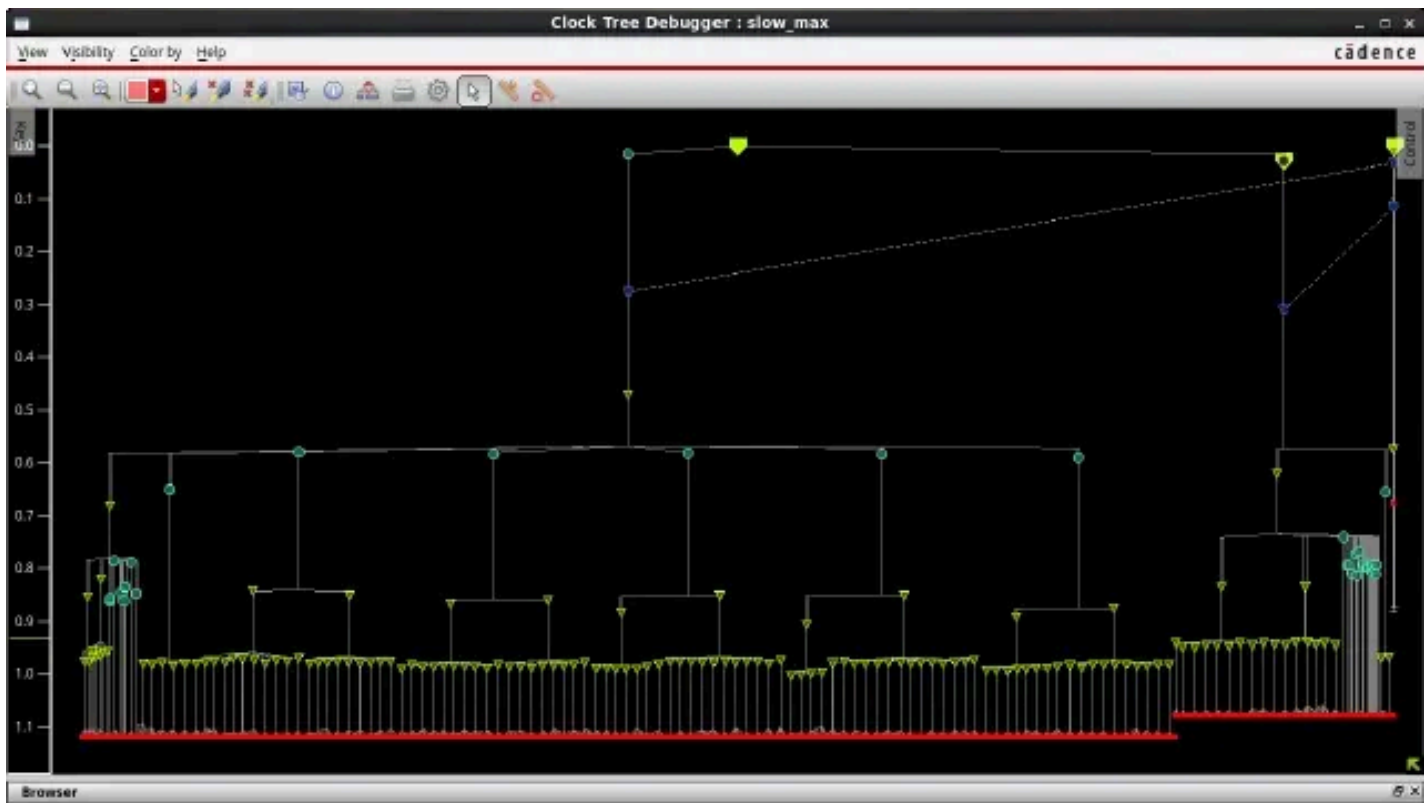
```
set_ccopt_property balance_mode trial
```

1. Run CCOpt-CTS with trial:

```
linux% innovus -log ./LOGS/Trial.log -init./SCRIPTS/runTrial.tcl
innovus> ctd_win
```


© 2017 Cadence Design Systems, Inc. All rights reserved worldwide.

Observe that most sinks are virtually delay-balanced. On the lower-left side, you can see a lot of sinks relating to my_clk. On the lower-right side, you can see a lot of sinks relating to div_clk and one ignore pin.




```
innovus> llength [get_ccopt_clock_tree_cells *]

185

innovus> get_cells -hier -filter "name !~ CTS_ccl* and name =~ CTS*"

mcore0/a0/CTS_ccd_BUF_div_clk_G0_L2_1

0x2b8e

innovus> sizeof_collection [get_cells  [get_ccopt_clock_tree_cells *] -
filter "name =~ CTS*"]

146
```


© 2017 Cadence Design Systems, Inc. All rights reserved worldwide.

```
innovus> source cell_cnt.tcl
TLATNTSCAX2 : 14
TLATNTSCAX20 : 3
TLATNTSCAX3 : 3
CLKBUFX20 : 99
TLATNTSCAX4 : 2
MX2X4 : 1
TLATNTSCAX6 : 8
TLATNTSCAX8 : 3
MX2X8 : 1
DFFX1 : 1
TLATNTSCAX12 : 1
CLKBUFX12 : 6
TLATNTSCAX16 : 2
CLKBUFX16 : 39
CLKBUFX8 : 2
```

Observe that the number of clock tree cells and their types after cluster CTS is equal to those after trial CTS.

If you compare the two timing reports, after cluster CTS and after trial CTS, you can see that the wire delays after trial CTS are increased because of the added virtual delays.

After cluster CTS:

```
innovus> report_timing -from clk -to proc0/rf0/u0/u1/rfd_reg_127_30/DFF/CK -unconstrained -net
Path 1:Endpoint:   proc0/rf0/u0/u1/rfd_reg_127_30/DFF/CK (v) (unconstrained output)
Beginpoint: clk                              (v) triggered by trailing
edge of 'my_clk'
Arrival Time                -0.051
Analysis View: func_slow_max
     + Drive Adjustment                 0.032
     + Source Insertion Delay          -1.022
     = Beginpoint Arrival Time         -0.991
     +-----------------------------------------------------------------------------------------------------------------------------------+
     |          Instance                |    Arc     |          Net          |    Cell    | Slew | Delay | Arrival | Required |
     |                                  |            |                       |            |      |       |  Time   |   Time   |
     |----------------------------------+------------+-----------------------+------------+------+-------+---------+----------|
     |                                  | clk v      | clk                   |            | 0.032|       | -0.991  |          |
     | CGIC_INST                        |            | clk                   | TLATNTSCAX6| 0.048| 0.015 | -0.975  |          |
     | CGIC_INST                        | CK v -> ECK v | clk_gate           | TLATNTSCAX6| 0.100| 0.225 | -0.750  |          |
     | clk_out_mux                      |            | clk_gate              | MX2X4      | 0.101| 0.005 | -0.745  |          |
     | clk_out_mux                      | A v -> Y v | clk2x                 | MX2X4      | 0.062| 0.186 | -0.559  |          |
     | CTS_ccl_BUF_my_clk_G1_L3_1       |            | clk2x                 | CLKBUFX20  | 0.062| 0.000 | -0.558  |          |
     | CTS_ccl_BUF_my_clk_G1_L3_1       | A v -> Y v | CTS_70                | CLKBUFX20  | 0.079| 0.096 | -0.463  |          |
     | CTS_ccl_BUF_my_clk_G1_L4_1       |            | CTS_70                | CLKBUFX20  | 0.085| 0.014 | -0.448  |          |
     | CTS_ccl_BUF_my_clk_G1_L4_1       | A v -> Y v | CTS_69                | CLKBUFX20  | 0.057| 0.100 | -0.349  |          |
     | proc0/CTS_ccl_BUF_my_clk_G1_L5_17|            | CTS_69                | CLKBUFX20  | 0.057| 0.002 | -0.347  |          |
     | proc0/CTS_ccl_BUF_my_clk_G1_L5_17| A v -> Y v | proc0/CTS_25          | CLKBUFX20  | 0.131| 0.121 | -0.225  |          |
     | proc0/rf0/u0/u1/CTS_ccl_BUF_my_clk_G1_L6_95 |  | proc0/CTS_25          | CLKBUFX8   | 0.134| 0.012 | -0.213  |          |
     | proc0/rf0/u0/u1/CTS_ccl_BUF_my_clk_G1_L6_95 | A v -> Y v | proc0/rf0/u0/u1/CTS_160 | CLKBUFX8 | 0.145| 0.160 | -0.053  |          |
     | proc0/rf0/u0/u1/rfd_reg_127_30/DFF |  | proc0/rf0/u0/u1/CTS_160 | SEDFFHQX1  | 0.145| 0.002 | -0.051  |          |
     +-----------------------------------------------------------------------------------------------------------------------------------+
```


© 2017 Cadence Design Systems, Inc. All rights reserved worldwide.

After trial CTS:

```
innovus> report_timing -from clk -to proc0/rf0/u0/u1/rfd_reg_127_30/DFF/CK -unconstrained -net
Path 1:Endpoint:   proc0/rf0/u0/u1/rfd_reg_127_30/DFF/CK (v) (unconstrained output)
Beginpoint: clk                          (v) triggered by trailing
edge of 'my_clk'
Arrival Time                  0.054
Analysis View: func_slow_max
     + Drive Adjustment                0.032
     + Source Insertion Delay         -1.071
     = Beginpoint Arrival Time        -1.039
```

* Virtual delay added

Instance	Arc	Net	Cell	Slew	Delay	Arrival Time	Required Time
	clk v	clk		0.032		-1.039	
CGIC_INST		clk	TLATNTSC[illegible]	0.048	0.015	-1.024	
CGIC_INST	CK v -> ECK v	clk_gate	TLATNTSCA[illegible]	0.109	0.225	-0.799	
clk_out_mux		clk_gate	MX2X4	0.101	0.005	-0.794	
clk_out_mux	A v -> Y v	clk2x	MX2X4	0.062	0.186	-0.608	
CTS_ccl_BUF_my_clk_G1_L3_1		clk2x	CLKBUFX20	0.062	0.000	-0.607	
CTS_ccl_BUF_my_clk_G1_L3_1	A v -> Y v	CTS_70	CLKBUFX20	0.079	0.096	-0.511	
CTS_ccl_BUF_CLOCK_NODE_UID_A34489		CTS_70	CLKBUFX20	0.765	0.113	-0.398	
CTS_ccl_BUF_CLOCK_NODE_UID_A34489	A v -> Y v	CTS_69	CLKBUFX20	0.057	0.100	-0.299	
proc0/CTS_ccl_BUF_CLOCK_NODE_UID_A34486		CTS_69	CLKBUFX20	0.051	0.042	-0.257	
proc0/CTS_ccl_BUF_CLOCK_NODE_UID_A34486	A v -> Y v	proc0/CTS_25	CLKBUFX20	0.131	0.121	-0.135	
proc0/rf0/u0/u1/CTS_ccl_BUF_CLOCK_NODE_UID_A34478		proc0/CTS_25	CLKBUFX8	0.134	0.021	-0.114	
proc0/rf0/u0/u1/CTS_ccl_BUF_CLOCK_NODE_UID_A34478	A v -> Y v	proc0/rf0/u0/u1/CTS_160	CLKBUFX8	0.145	0.160	0.046	
proc0/rf0/u0/u1/rfd_reg_127_30/DFF		proc0/rf0/u0/u1/CTS_160	SEDFFHQX1	0.145	0.008	0.054	

Exit the Innovus session and view the following Runtime Summary after trial CTS in the *./LOGS/Trial.log* file:

```
Runtime Summary:
================

-----------------------------------------------------------------------------------------------------
wall    % time  children  called  name
-----------------------------------------------------------------------------------------------------
136.90  100.00   136.90     0
136.90  100.00   136.29     1     Runtime
  0.45    0.33     0.45     1     CCOpt::Phase::Initialization
  0.45    0.33     0.45     1         Check Prerequisites
  0.45    0.33     0.00     1         Leaving CCOpt scope - CheckPlace
  3.73    2.73     0.00     1         Leaving CCOpt scope - optDesignGlobalRouteStep
  4.98    3.64     0.00     1         Validating CTS configuration
  0.86    0.63     0.00     1         Preparing To Balance
102.41   74.81   102.41     1     CCOpt::Phase::Clustering
 40.45   29.54    34.68     1       Stage::DRV Fixing
 30.21   22.07     2.11     1         Clustering
  2.11    1.54     0.00     1         Leaving CCOpt scope - RefinePlacement
  1.30    0.95     0.00     1         Leaving CCOpt scope
  2.04    1.49     0.00     1         Fixing clock tree slew time and max cap violations
  1.12    0.82     0.00     1         Fixing clock tree slew time and max cap violations - detailed pass
 61.97   45.27    61.86     1       Stage::Insertion Delay Reduction
  0.85    0.62     0.00     1         Removing unnecessary root buffering
  0.69    0.50     0.00     1         Removing unconstrained drivers
  0.76    0.55     0.00     1         Reducing insertion delay 1
  2.46    1.79     0.00     1         Removing longest path buffering
 57.11   41.71     0.00     1         Reducing insertion delay 2
 10.91    7.97     4.73     1     CCOpt::Phase::Implementation
  1.26    0.92     0.00     1         Leaving CCOpt scope
  3.46    2.53     0.00     1         Adding trial balancing virtual delays
 12.93    9.45     0.00     1         Tidy Up And Update Timing
-----------------------------------------------------------------------------------------------------
```


© 2017 Cadence Design Systems, Inc. All rights reserved worldwide.

Observe the additional step named “Adding trial balancing virtual delays”, which was not in the `./LOGS/Cluster.log` file.

3-5. Running CCOpt-CTS with full mode

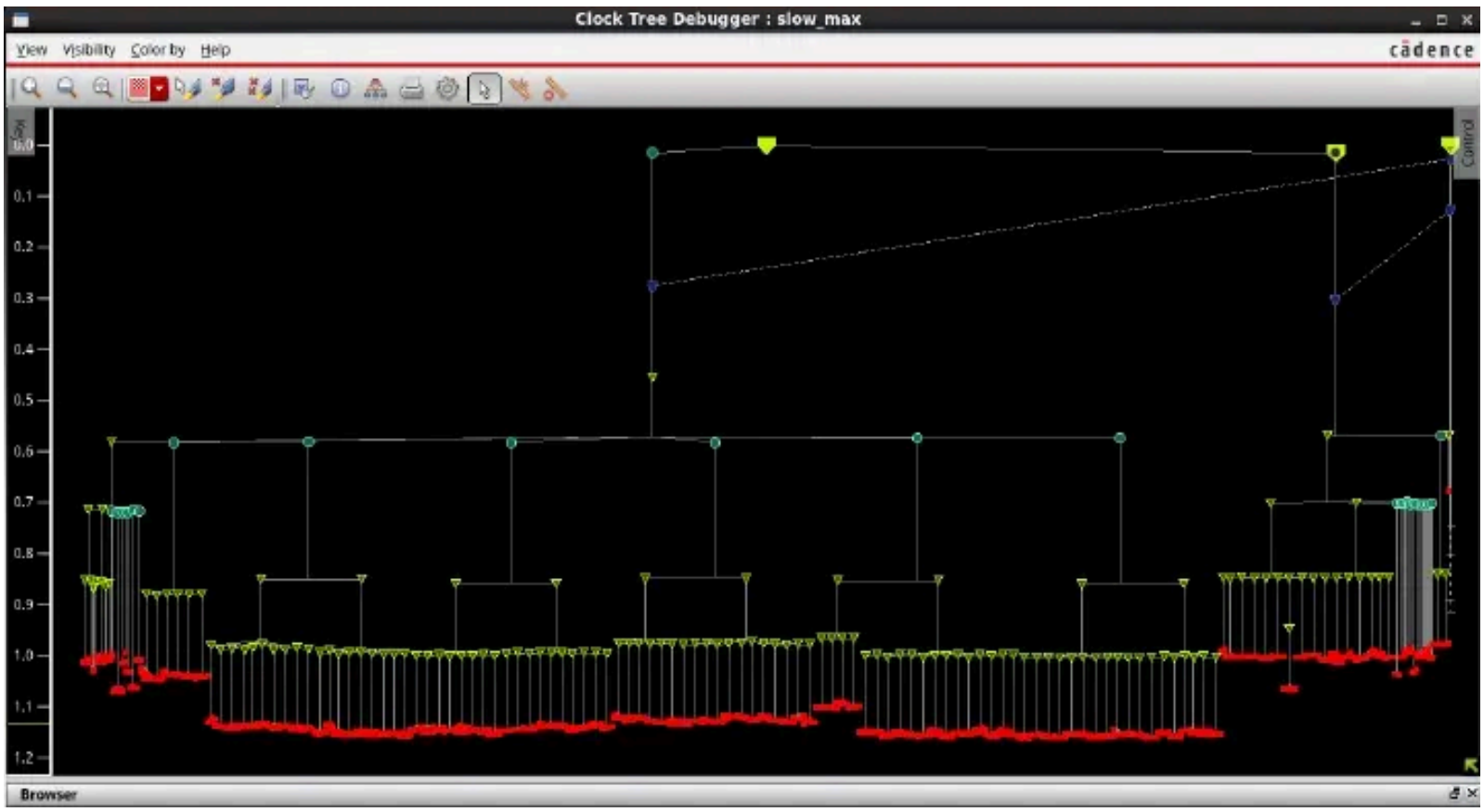
The following command enables you to run CCOpt-CTS with full mode:

```
set_ccopt_property balance_mode full
```

1. Run CCOpt-CTS with full:

   ```
   linux% innovus -log ./LOGS/CTS.log -
   init./SCRIPTS/runFull.tcl
   ```

   Run *`ctd_win`*.



© 2017 Cadence Design Systems, Inc. All rights reserved worldwide.

Exit the Innovus session, then open the./LOGS/CTS.log file and view the following Runtime Summary. Try to locate each step in the log file.

```
Runtime Summary:
================
-----------------------------------------------------------------------------------------------
wall    % time  children  called  name
-----------------------------------------------------------------------------------------------
367.96  100.00   367.96     0
367.96  100.00   367.40     1     Runtime
  0.44    0.12     0.44     1     CCOpt::Phase::Initialization
  0.44    0.12     0.44     1         Check Prerequisites
  0.44    0.12     0.00     1         Leaving CCOpt scope - CheckPlace
  3.05    0.83     0.00     1         Leaving CCOpt scope - optDesignGlobalRouteStep
  4.65    1.26     0.00     1         Validating CTS configuration
  0.57    0.16     0.00     1         Preparing To Balance
102.53   27.86   102.53     1     CCOpt::Phase::Clustering
 40.82   11.09    34.56     1       Stage::DRV Fixing
 30.17    8.20     2.14     1         Clustering
  2.14    0.58     0.00     1         Leaving CCOpt scope - RefinePlacement
  1.27    0.35     0.00     1         Leaving CCOpt scope
  2.02    0.55     0.00     1         Fixing clock tree slew time and max cap violations
  1.09    0.30     0.00     1         Fixing clock tree slew time and max cap violations - detailed pass
 61.71   16.77    61.59     1       Stage::Insertion Delay Reduction
  0.86    0.23     0.00     1         Removing unnecessary root buffering
  0.73    0.20     0.00     1         Removing unconstrained drivers
  0.82    0.22     0.00     1         Reducing insertion delay 1
  2.53    0.69     0.00     1         Removing longest path buffering
 56.66   15.40     0.00     1         Reducing insertion delay 2
243.75   66.25   235.89     1     CCOpt::Phase::Implementation
 15.81    4.30    15.81     1       Stage::Reducing Power
 15.07    4.10     0.00     1         Reducing clock tree power 1
  0.74    0.20     0.00     1         Reducing clock tree power 2
 15.38    4.18    14.51     1       Stage::Balancing
 13.49    3.67    10.92     1         Approximately balancing fragments step
 10.92    2.97     0.00     1         Approximately balancing fragments bottom up
  1.02    0.28     0.00     1         Improving fragments clock skew
  2.18    0.59     0.00     1         Approximately balancing step
  0.74    0.20     0.00     1         Fixing clock tree overload
  0.85    0.23     0.00     1         Approximately balancing paths
  1.26    0.34     0.00     1         Leaving CCOpt scope
 18.20    4.95    18.20     1       Stage::Polishing
  1.27    0.35     0.00     1         Improving clock skew
 15.89    4.32     0.00     1         Reducing clock tree power 3
  1.04    0.28     0.00     1         Improving insertion delay
101.72   27.64    85.76     1       Stage::Routing
  2.52    0.69     0.00     1         Leaving CCOpt scope - ClockRefiner
 81.77   22.22     0.00     1         Leaving CCOpt scope - NanoRouter
  1.47    0.40     0.00     1         Leaving CCOpt scope
 79.75   21.67    57.55     1       Stage::PostConditioning
 14.85    4.04     0.00     3         Leaving CCOpt scope
  3.24    0.88     0.00     1         Fixing DRVs
 36.96   10.05     0.00     1         Buffering to fix DRVs
  2.50    0.68     2.49     1         Refining placement
  2.49    0.68     0.00     1         Leaving CCOpt scope - ClockRefiner
 12.40    3.37     0.00     1         Tidy Up And Update Timing
-----------------------------------------------------------------------------------------------
```

Observe the additional steps after Balancing, such as Polishing, Routing, and PostConditioning.



© 2017 Cadence Design Systems, Inc. All rights reserved worldwide.

Module 4: **Running and Analyzing the CCOpt Results**

© 2017 Cadence Design Systems, Inc. All rights reserved worldwide.

Lab 4-1 Running CCOpt and Analyzing its results

Objective: Run CCOpt and analyze its results.

4-1 Analyzing the Log file

The following command enables you to run CCOpt:

```
set_ccopt_property balance_mode full
setOptMode -usefulSkewCCOpt medium
ccopt_design
```

1. Run CCOpt:

```
linux% innovus -log ./LOGS/CCOpt.log -
init./SCRIPTS/runCCOpt.tcl

innovus> ctd_win
```

Try to read the `./LOGS/CCOpt.log./` log file

This section highlights the parts of the log file you can review to analyze the CCOpt run.

With the log file open, search for the following sections:

Section	Description
Initial Summary	Initial timing before optimization
Skew group insertion delays	Insertion delay for each skew group
Guided max path lengths	Breakdown of the route guide lengths
Deviation of routing from guided max path lengths	Shows how the actual routing deviates from the route guides
Top 10 notable deviations of routed length from guided length	Specific nets whose length deviates most from their route guides
Clock DAG stats at the end of CTS	Count and area of cells used in the initial clock tree
Clock DAG capacitances at the end of CTS	Wire and Gate capacitance of the initial clock tree

© 2017 Cadence Design Systems, Inc. All rights reserved worldwide.

Clock DAG primary half-corner transition distribution at the end of CTS	Max/Min trunk and leaf slew Target, Count, and Distribution at primary half-corner
Skew group summary at the end of CTS	Max/Min ID and skew information for all skew groups
GigaOpt + CCOpt summary information	WNS, TNS, and runtime summary of GigaOpt + CCOpt optimization
optDesign Final Summary	Final timing results including WNS, TNS, and DRV information

Note that the green highlighted items are only for CCOpt flow.

© 2017 Cadence Design Systems, Inc. All rights reserved worldwide.

View the following Runtime Summary in the *./LOGS/CCOpt.log* file:

```
Runtime Summary:
================

-----------------------------------------------------------------------------------------------------
wall    % time  children  called  name
-----------------------------------------------------------------------------------------------------
695.79  100.00   695.79     0
695.79  100.00   695.64     1     Runtime
  5.46    0.78     5.46     1     CCOpt::Phase::Initialization
  5.46    0.78     5.45     1         Check Prerequisites
  0.66    0.10     0.00     1         Leaving CCOpt scope - CheckPlace
  4.79    0.69     0.00     1         Validating CTS configuration
690.18   99.19   425.48     1     External::optDesign
  3.01    0.43     0.00     3         Validating CTS configuration
  4.69    0.67     0.00     2         Preparing To Balance
 80.82   11.62    80.82     1     CCOpt::Phase::Clustering
 41.94    6.03    35.35     1       Stage::DRV Fixing
 31.08    4.47     2.19     1         Clustering
  2.19    0.32     0.00     1         Leaving CCOpt scope - RefinePlacement
  1.30    0.19     0.00     1         Leaving CCOpt scope
  1.88    0.27     0.00     1         Fixing clock tree slew time and max cap violations
  1.08    0.16     0.00     1         Fixing clock tree slew time and max cap violations - detailed pass
 38.89    5.59    38.77     1       Stage::Insertion Delay Reduction
  0.71    0.10     0.00     1         Removing unnecessary root buffering
  0.72    0.10     0.00     1         Removing unconstrained drivers
  0.92    0.13     0.00     1         Reducing insertion delay 1
  1.95    0.28     0.00     1         Removing longest path buffering
 34.47    4.95     0.00     1         Reducing insertion delay 2
296.96   42.68   282.50     2     CCOpt::Phase::Implementation
  2.62    0.38     0.00     2         Leaving CCOpt scope
 34.39    4.94     0.00     5         Leaving CCOpt scope
  5.61    0.81     0.00     1         Fixing clock tree slew time and max cap violations
 57.23    8.23    56.28     1       Stage::Balancing
 18.01    2.59     0.00     1         Reducing clock tree power 1
 13.25    1.90     0.00     1         Reducing clock tree power 2
 15.83    2.28    13.99     1         Approximately balancing fragments step
 13.99    2.01     0.00     1         Approximately balancing fragments bottom up
  9.18    1.32     0.00     1         Improving fragments clock skew
  1.30    0.19     0.00     1         Approximately balancing step
  0.67    0.10     0.00     1         Fixing clock tree overload
  5.80    0.83     0.00     1         Approximately balancing paths
61.29    8.81    61.29     1       Stage::Polishing
 31.42    4.52     0.00     1         Improving clock skew
 17.24    2.48     0.00     1         Reducing clock tree power 3
 11.78    1.69     0.00     1         Reducing total underdelay
  0.86    0.12     0.00     1         Improving insertion delay
100.82   14.49    84.31     1       Stage::Routing
  2.82    0.41     0.00     1         Leaving CCOpt scope - ClockRefiner
 80.18   11.52     0.00     1         Leaving CCOpt scope - NanoRouter
  1.31    0.19     0.00     1         Leaving CCOpt scope
 52.77    7.58    28.79     1       Stage::PostConditioning
 15.42    2.22     0.00     3         Leaving CCOpt scope
  3.21    0.46     0.00     1         Fixing DRVs
  7.45    1.07     0.00     1         Buffering to fix DRVs
  2.70    0.39     2.69     1         Refining placement
  2.69    0.39     0.00     1         Leaving CCOpt scope - ClockRefiner
-----------------------------------------------------------------------------------------------------
```


© 2017 Cadence Design Systems, Inc. All rights reserved worldwide.

4-2 CCOpt Analysis Commands

This section introduces you to some common commands to report the results of CCOpt.

- **Skew groups** specify which sinks should be balanced together (balancing of the clock trees and balancing between the clock trees).

1. Run `report_ccopt_skew_groups` to report the minimum and maximum paths and skews for all skew groups.

Review the output of these sections with the `report_ccopt_skew_groups` command:

- **Skew Group Structure**: Information on the number of sources and sinks for each skew group.
- **Skew Group Summary**: Insertion Delay (ID) and skew information for each skew group respective to each delay corner.
- **Skew Group Min/Max path pins**: End points with a minimum and maximum insertion delay for each skew group respective to each delay corner. This is followed by the detailed path information.

```
innovus> report_ccopt_skew_groups

Skew Group Structure:
=====================

------------------------------------------------------------------------------
Skew Group                          Sources    Constrained Sinks    Ignore Sinks
------------------------------------------------------------------------------
div_clk/functional_func_slow_max        1              1922               1
my_clk/functional_func_slow_max         1             11536               1
test_clk/functional_func_slow_max       1             11536               1
------------------------------------------------------------------------------

Skew Group Summary:
===================

------------------------------------------------------------------------------------------------------------------------------------------------------
Timing Corner          Skew Group                          ID Target    Min ID    Max ID    Avg ID    Std.Dev. ID    Skew Target Type    Skew Target    Skew     Skew window occupancy
------------------------------------------------------------------------------------------------------------------------------------------------------
slow_max:setup.early   div_clk/functional_func_slow_max         -       0.788     0.954     0.819        0.013        ignored                -       0.166              -
                       my_clk/functional_func_slow_max          -       1.075     1.286     1.176        0.036        ignored                -       0.211              -
                       test_clk/functional_func_slow_max        -       0.776     1.035     0.917        0.052        ignored                -       0.259              -
slow_max:setup.late    div_clk/functional_func_slow_max      none       0.788     0.954     0.820        0.013        explicit            0.200       0.166    100% {0.788, 0.954}
                       my_clk/functional_func_slow_max       none       1.083     1.294     1.183        0.036        explicit           *0.200       0.210    99.5% {1.085, 1.285}
                       test_clk/functional_func_slow_max     none       0.791     1.049     0.930        0.052        explicit           *0.200       0.258    99.2% {0.791, 0.991}
fast_min:hold.early    div_clk/functional_func_slow_max         -       0.259     0.343     0.288        0.010        ignored                -       0.084              -
                       my_clk/functional_func_slow_max          -       0.366     0.471     0.424        0.016        ignored                -       0.105              -
                       test_clk/functional_func_slow_max        -       0.272     0.393     0.343        0.021        ignored                -       0.121              -
fast_min:hold.late     div_clk/functional_func_slow_max         -       0.259     0.343     0.288        0.010        explicit            0.200       0.084    100% {0.259, 0.343}
                       my_clk/functional_func_slow_max          -       0.372     0.477     0.438        0.016        explicit            0.200       0.105    100% {0.372, 0.477}
                       test_clk/functional_func_slow_max        -       0.281     0.402     0.352        0.021        explicit            0.200       0.120    100% {0.281, 0.402}
------------------------------------------------------------------------------------------------------------------------------------------------------

* - indicates that target was not met.
```


© 2017 Cadence Design Systems, Inc. All rights reserved worldwide.

```
Skew Group Min/Max path pins:
=============================

-------------------------------------------------------------------------------------------------------
Timing Corner           Skew Group                          Min ID    PathID    Max ID    PathID
-------------------------------------------------------------------------------------------------------
slow_max:setup.early    div_clk/functional_func_slow_max    0.788        1      0.954        2
-    min mcore0/mctrl0/r_reg_mcfg1_romwrite/DFF/CK
-    max mcore0/mctrl0/r_reg_ramoen_4/DFF/CK
                        my_clk/functional_func_slow_max     1.075        9      1.286       10
-    min mcore0/mctrl0/r_reg_mcfg1_romwrite/DFF/CK
-    max proc0/rf0/u0/u1/rfd_reg_119_28/DFF/CK
                        test_clk/functional_func_slow_max   0.776       17      1.035       18
-    min mcore0/mctrl0/r_reg_mcfg1_romwrite/DFF/CK
-    max proc0/rf0/u0/u1/rfd_reg_119_28/DFF/CK
slow_max:setup.late     div_clk/functional_func_slow_max    0.788        3      0.954        4
-    min mcore0/mctrl0/r_reg_mcfg1_romwrite/DFF/CK
-    max mcore0/mctrl0/r_reg_ramoen_4/DFF/CK
                        my_clk/functional_func_slow_max     1.083       11      1.294       12
-    min mcore0/mctrl0/r_reg_mcfg1_romwrite/DFF/CK
-    max proc0/rf0/u0/u1/rfd_reg_119_28/DFF/CK
                        test_clk/functional_func_slow_max   0.791       19      1.049       20
-    min mcore0/mctrl0/r_reg_mcfg1_romwrite/DFF/CK
-    max proc0/rf0/u0/u1/rfd_reg_119_28/DFF/CK
fast_min:hold.early     div_clk/functional_func_slow_max    0.259        5      0.343        6
-    min mcore0/mctrl0/r_reg_mcfg1_romwrite/DFF/CK
-    max mcore0/mctrl0/r_reg_ramoen_4/DFF/CK
                        my_clk/functional_func_slow_max     0.366       13      0.471       14
-    min mcore0/mctrl0/r_reg_mcfg1_romwrite/DFF/CK
-    max proc0/rf0/u0/u1/rfd_reg_119_27/DFF/CK
                        test_clk/functional_func_slow_max   0.272       21      0.393       22
-    min mcore0/mctrl0/r_reg_mcfg1_romwrite/DFF/CK
-    max proc0/rf0/u0/u1/rfd_reg_119_27/DFF/CK
fast_min:hold.late      div_clk/functional_func_slow_max    0.259        7      0.343        8
-    min mcore0/mctrl0/r_reg_mcfg1_romwrite/DFF/CK
-    max mcore0/mctrl0/r_reg_ramoen_4/DFF/CK
                        my_clk/functional_func_slow_max     0.372       15      0.477       16
-    min mcore0/mctrl0/r_reg_mcfg1_romwrite/DFF/CK
-    max proc0/rf0/u0/u1/rfd_reg_119_27/DFF/CK
                        test_clk/functional_func_slow_max   0.281       23      0.402       24
-    min mcore0/mctrl0/r_reg_mcfg1_romwrite/DFF/CK
-    max proc0/rf0/u0/u1/rfd_reg_119_27/DFF/CK
-------------------------------------------------------------------------------------------------------
```

…

- **Clock trees** define the connectivity between cells and sinks.

2. Run `report_ccopt_clock_trees` to report the slew, area, and buffer count for each clock tree:

innovus> report_ccopt_clock_trees

© 2017 Cadence Design Systems, Inc. All rights reserved worldwide.

```
Clock DAG stats:
================

-------------------------------------------------------------
Cell type                 Count    Area       Capacitance
-------------------------------------------------------------
Buffers                     147    1045.152       0.449
Inverters                     0       0.000       0.000
Integrated Clock Gates       34     277.020       0.021
Non-Integrated Clock Gates    0       0.000       0.000
Clock Logic                   2       8.892       0.003
All                         183    1331.064       0.473
-------------------------------------------------------------


Clock DAG wire lengths:
=======================

--------------------
Type     Wire Length
--------------------
Top            0.000
Trunk      11174.580
Leaf       65799.560
Total      76974.140
--------------------


Clock DAG capacitances:
=======================

---------------------------------
Type     Gate     Wire      Total
---------------------------------
Top      0.000     0.000     0.000
Trunk    0.466     1.124     1.590
Leaf     7.201     8.949    16.151
Total    7.667    10.073    17.741
---------------------------------


Clock DAG sink capacitances:
============================

-----------------------------------------------------------
Count    Total    Average    Std. Dev.    Min      Max
-----------------------------------------------------------
11537    7.194     0.001       0.003      0.001    0.150
-----------------------------------------------------------


...
```


© 2017 Cadence Design Systems, Inc. All rights reserved worldwide.

Review this output with the `report_ccopt_clock_trees` command.

- **Worst Chain** is reported from time-to-time in the log during CCOpt, and an examination of the log may help identify the reasons limiting timing optimization. In addition, the `report_ccopt_worst_chain` command can be used to report the worst timing chain after `ccopt_design` has completed. However, note that this will reflect the current worst chain and not the worst chain during optimization.

3. Run `report_ccopt_worst_chain` to report the worst chain for the design.

```
innovus> report_ccopt_worst_chain

Enabling path groups and categories...

Enabling path groups and categories done.

Creating worst chain report...

Creating worst chain report done.
```


© 2017 Cadence Design Systems, Inc. All rights reserved worldwide.

```
x clk_en
|   slack 2.658ns .../clk_en -> .../E (distance: 304.085um)
|                 delays=(launch: 0.000ns, datapath: 0.099ns, capture: -1.143ns, adjust: 3.900ns)
|                 launch no clock
|                 capture clock my_clk in analysis view func_slow_max
|                 path group default
g CGIC_INST
|     .../CK @+0.015ns constraint=(-0.000ns,+0.000ns) chosen=(-0.097ns,+0.000ns)
|           location=(503.100,402.515) slew=(launch: 0.045ns, capture: 0.032ns)
|     clock gate above
o proc0/cmem0/dtags0/u0/id0
|     .../CK2 @+0.716ns constraint=(-0.150ns,+0.579ns) chosen=(-0.192ns,+0.035ns)
|           location=(723.635,162.785) slew=(launch: 0.132ns, capture: 0.132ns)
|   slack 0.136ns .../Q1[23] -> .../D (distance: 635.040um)
|                 delays=(launch: 0.023ns, datapath: 3.759ns, capture: -0.040ns, adjust: 3.958ns)
|                 launch clock my_clk in analysis view func_slow_max
|                 capture clock test_clk in analysis view func_slow_max
|                 path group reg2reg
o proc0/c0/dcache0/r_reg_req/DFF
|     .../CK @+0.716ns constraint=(-0.175ns,+0.579ns) chosen=(-0.079ns,+0.000ns)
|           location=(383.700,328.795) slew=(launch: 0.136ns, capture: 0.136ns)
|   slack 0.854ns .../Q -> .../D (distance: 368.320um)
|                 delays=(launch: 0.018ns, datapath: 3.127ns, capture: -0.101ns, adjust: 4.099ns)
|                 launch clock my_clk in analysis view func_slow_max
|                 capture clock test_clk in analysis view func_slow_max
|                 path group reg2reg
o mcore0/ahb0/r_reg_hslave_1/DFF
|     .../CK @+0.513ns constraint=(-0.164ns,+0.661ns) chosen=(-0.173ns,+0.038ns)
|           location=(217.500,319.865) slew=(launch: 0.139ns, capture: 0.139ns)
|   slack 0.196ns .../Q -> .../D (distance: 505.685um)
|                 delays=(launch: 0.006ns, datapath: 3.706ns, capture: -0.044ns, adjust: 3.951ns)
|                 launch clock div_clk in analysis view func_slow_max
|                 capture clock test_clk in analysis view func_slow_max
|                 path group reg2reg
o proc0/iu0/de_reg_inst_22/DFF
|     .../CK @+0.716ns constraint=(-0.137ns,+0.579ns) chosen=(-0.149ns,+0.002ns)
|           location=(418.100,366.225) slew=(launch: 0.128ns, capture: 0.128ns)
|   slack 0.038ns .../Q -> .../D (distance: 762.370um)
|                 delays=(launch: 0.014ns, datapath: 3.898ns, capture: -0.008ns, adjust: 3.958ns)
|                 launch clock my_clk in analysis view func_slow_max
|                 capture clock test_clk in analysis view func_slow_max
|                 path group reg2reg
o proc0/iu0/ex_reg_rs1data_1/DFF
|     .../CK @+0.716ns constraint=(-0.091ns,+0.579ns) chosen=(-0.016ns,+0.020ns)
|           location=(475.900,439.755) slew=(launch: 0.127ns, capture: 0.127ns)
|   *WNS* 0.007ns .../Q -> .../D (distance: 297.740um)
|                 delays=(launch: 0.050ns, datapath: 3.892ns, capture: -0.008ns, adjust: 3.958ns)
|                 launch clock my_clk in analysis view func_slow_max
|                 capture clock test_clk in analysis view func_slow_max
|                 path group reg2reg
o proc0/iu0/ex_reg_rs2data_22/DFF
|     .../CK @+0.716ns constraint=(-0.096ns,+0.579ns) chosen=(-0.020ns,+0.221ns)
|           location=(493.900,388.265) slew=(launch: 0.127ns, capture: 0.127ns)
|   slack 0.382ns .../Q -> .../D (distance: 235.890um)
|                 delays=(launch: 0.050ns, datapath: 3.884ns, capture: -0.008ns, adjust: 3.958ns)
|                 launch clock my_clk in analysis view func_slow_max
|                 capture clock test_clk in analysis view func_slow_max
|                 path group reg2reg
-o proc0/iu0/ex_reg_rs1data_31/DFF
|     .../CK @+0.716ns constraint=(-0.096ns,+0.579ns) chosen=(-0.015ns,+0.311ns)
|           location=(497.900,393.395) slew=(launch: 0.127ns, capture: 0.127ns)
|   slack 0.587ns .../Q -> .../D (distance: 297.830um)
|                 delays=(launch: 0.050ns, datapath: 3.855ns, capture: -0.009ns, adjust: 3.958ns)
|                 launch clock my_clk in analysis view func_slow_max
|                 capture clock test_clk in analysis view func_slow_max
|                 path group reg2reg
o proc0/iu0/ex_reg_rs1data_23/DFF
|     .../CK @+0.716ns constraint=(-0.096ns,+0.579ns) chosen=(-0.039ns,+0.150ns)
|           location=(511.900,402.325) slew=(launch: 0.127ns, capture: 0.127ns)
|   slack 0.094ns .../Q -> .../D (distance: 231.580um)
|                 delays=(launch: 0.049ns, datapath: 3.885ns, capture: -0.008ns, adjust: 3.958ns)
|                 launch clock my_clk in analysis view func_slow_max
|                 capture clock test_clk in analysis view func_slow_max
|                 path group reg2reg
-o proc0/iu0/ex_reg_rs1data_31/DFF
```

1

© 2017 Cadence Design Systems, Inc. All rights reserved worldwide.

In this report:

- 'x' in the row "x clk_en" indicates a pin that cannot be adjusted because it is outside the defined clock trees. This is commonly an IO pin. clk_en is an input port in the Leon design.

- 'g' in the row "g CGIC_INST" indicates a clock gate. CGIC_INST is a clock gating cell in the design.

- 'o' in the row "o proc0/cmem0/dtags0/u0/id0" indicates a clock tree sink that can be skewed.

Note that this command is not needed in the CCOpt-CTS flow. Review the information output with the report_ccopt_worst_chain command. You can reference the Reporting on the Worst Chain section of the *Innovus Implementation System User Guide.*

4-3 CCOpt Clock Tree Debugger (CTD)

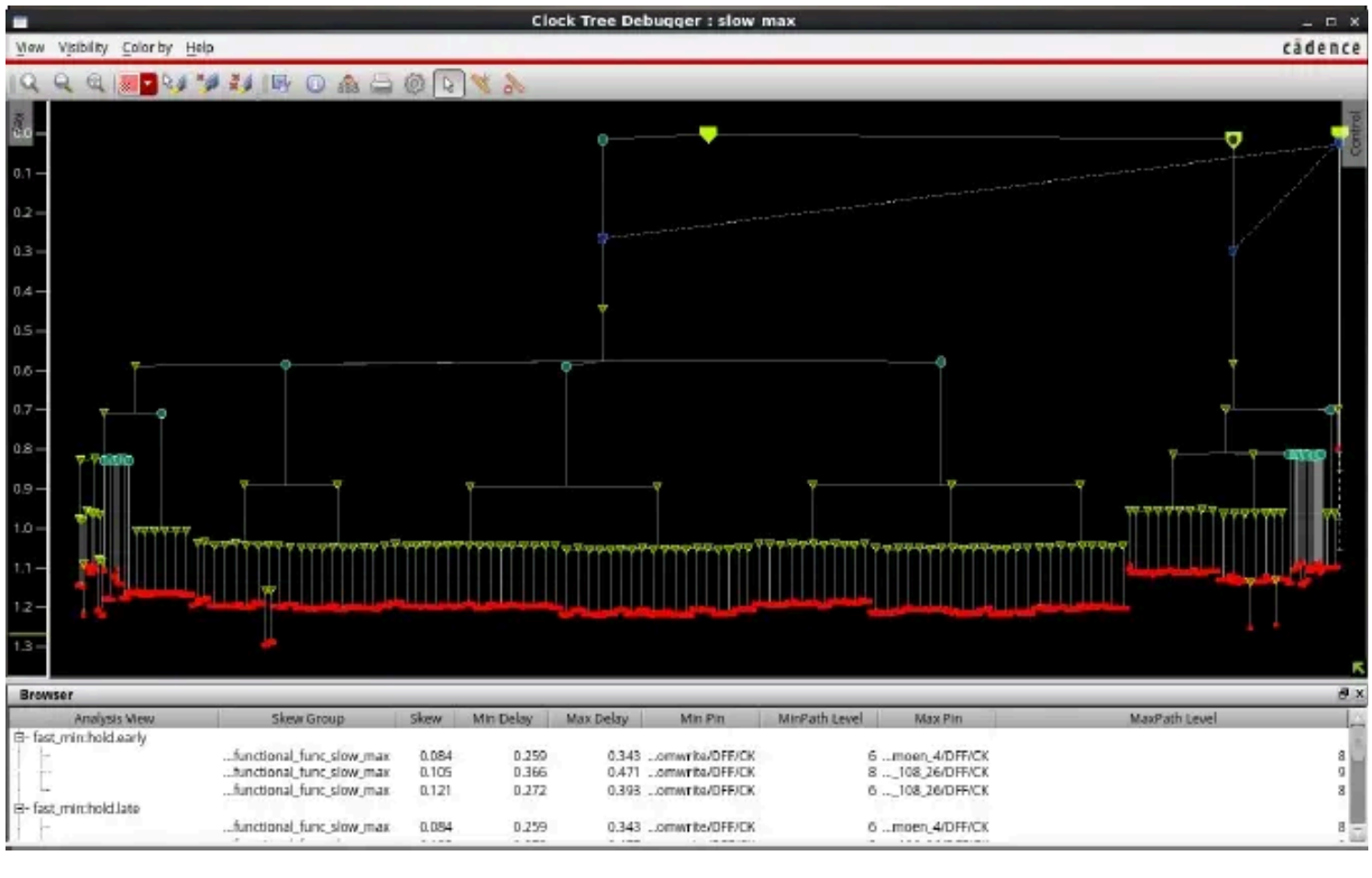
The CCOpt Clock Tree Debugger is a graphical tool for analyzing and debugging the clock tree results. It comes under the **Clock** menu and becomes available once the CCOpt clock tree constraints are defined (that is, after running create_ccopt_clock_tree_spec).

1. Restore the *DBS/postcts.enc* file that was saved in section 4-1, if you exited the Innovus session after that.

© 2017 Cadence Design Systems, Inc. All rights reserved worldwide.

2. Select Clock > CCOpt Clock Tree Debugger.

   The CCOpt Clock Tree Debugger appears.



   The debugger shows the logic in the clock tree with the clock roots at the top and the leaf cells at the bottom. Analysis is performed by coloring the clock tree based on selected criteria such as objects, paths, groups, and so on.

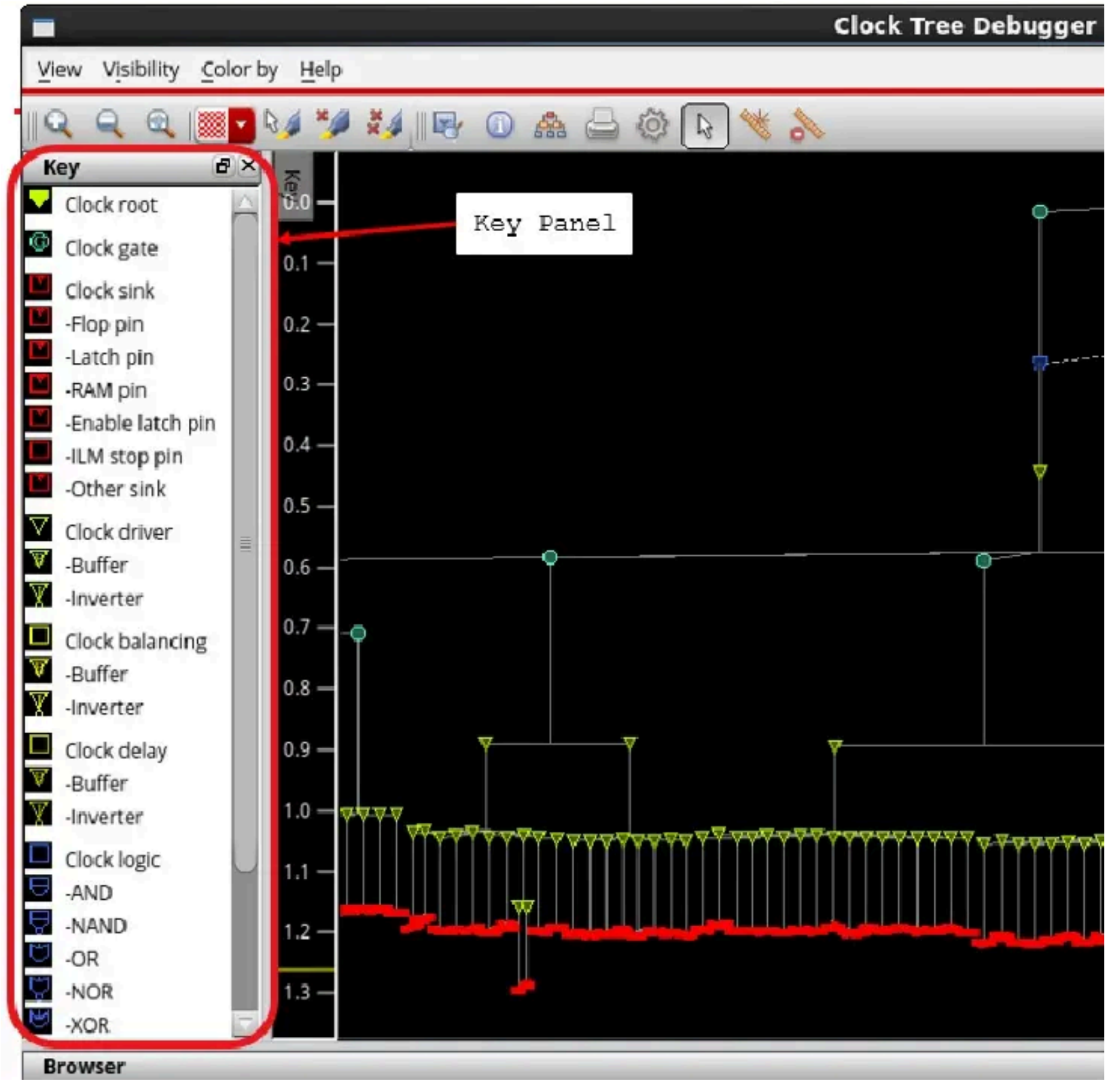
Following are the basics to navigate the GUI. These are the same as the Innovus Implementation System GUI.

- Use **z** and **Shift+z** (or **Z**) to zoom-in and zoom-out, respectively.
- Click and drag using the right mouse button (RMB) to zoom to a specific area.
- Press **F12** to toggle through the three brightness levels
- Use **f** to fit the design.
- Open the **World View** by clicking the green arrow that is in the lower-right corner. You can click and drag using the RMB within the World View to zoom to a specific area.
- Hover over an instance to display the properties of that instance.

The **Key Panel** provides a key that shows what each color represents. The Key Panel contents will vary depending on the type of data that is colored.

© 2017 Cadence Design Systems, Inc. All rights reserved worldwide.

3. Display the Key Panel by selecting **View > Key Panel** or by selecting the Key tab in the upper-left corner.



Initially, the clock tree is colored based on the cell type. Use the Key Panel to determine the symbols that represent each cell type.

The **Visibility** menu filters which colored objects are to be made visible.

© 2017 Cadence Design Systems, Inc. All rights reserved worldwide.

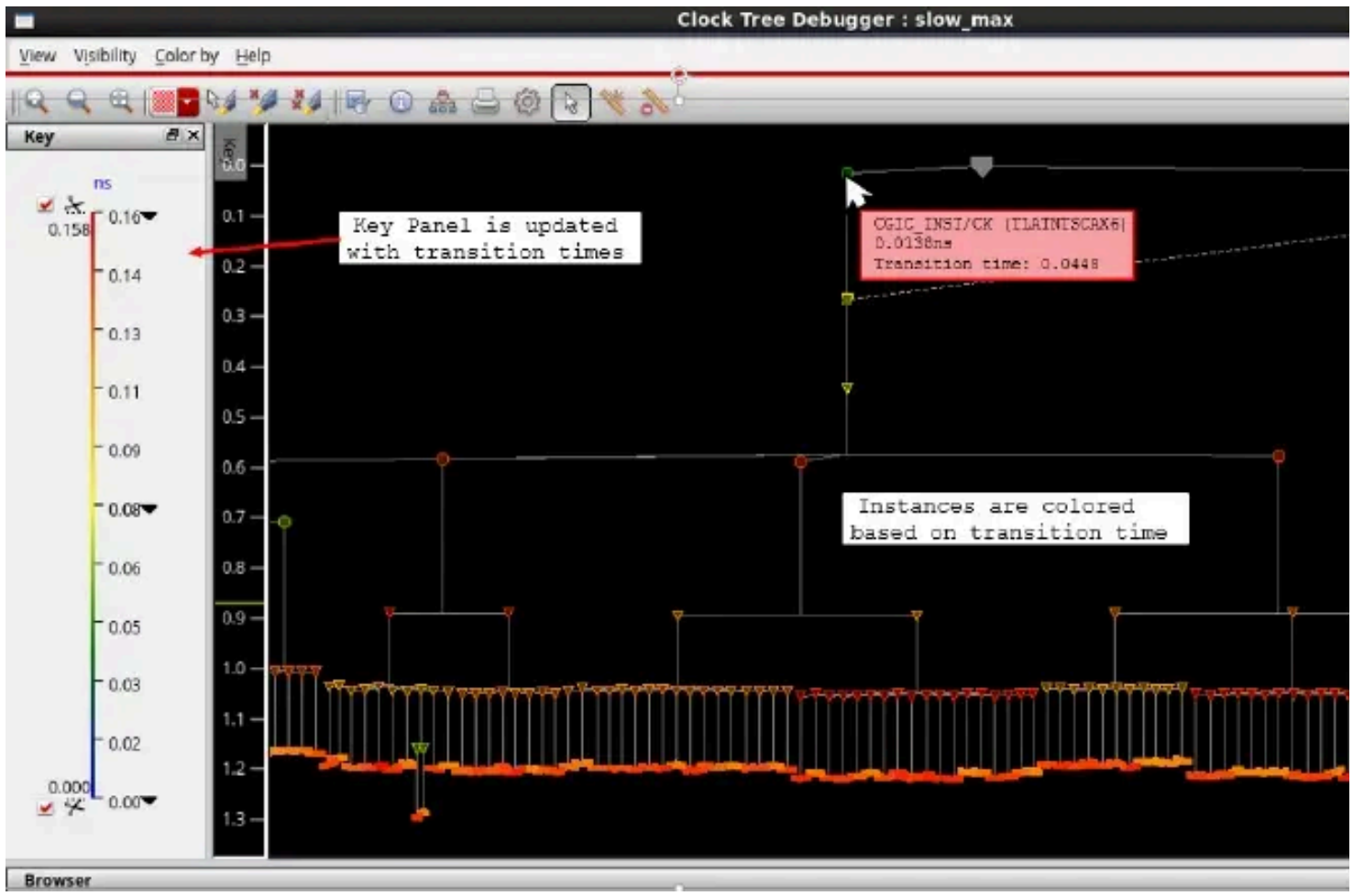
4. Enable the check box next to **Visibility > Cell type > Clock sink** to disable the coloring of all the clock sinks.



Observe that the clock sinks at the bottom of the tree are no longer colored.

5. Select **Color by > Transition Time** to color the clock tree based on the transition time arrival at each cell.

© 2017 Cadence Design Systems, Inc. All rights reserved worldwide.

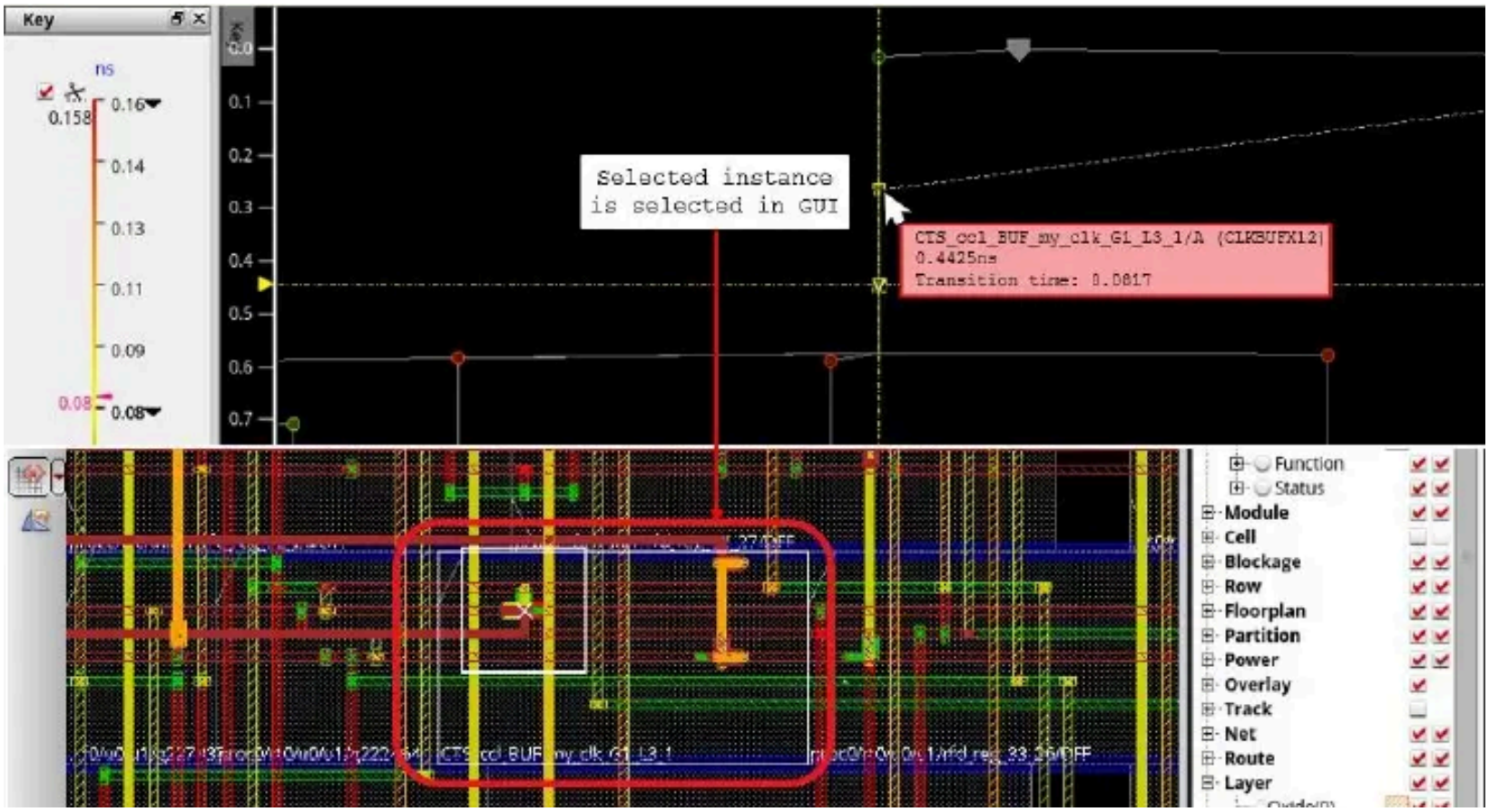


© 2017 Cadence Design Systems, Inc. All rights reserved worldwide.

Observe that the Key Panel changes to show a gradient of colors representing the transition times. You can hover over an instance to see its transition time and confirm that its color corresponds to the color in the Key Panel.

When you select a net or instance in the debugger, the Innovus Implementation System GUI will automatically select and zoom to that object. This cross probing makes it easy to determine the location of the object in the floorplan.

6. Select an instance or net in the debugger and observe its selection in the Innovus Implementation System GUI.

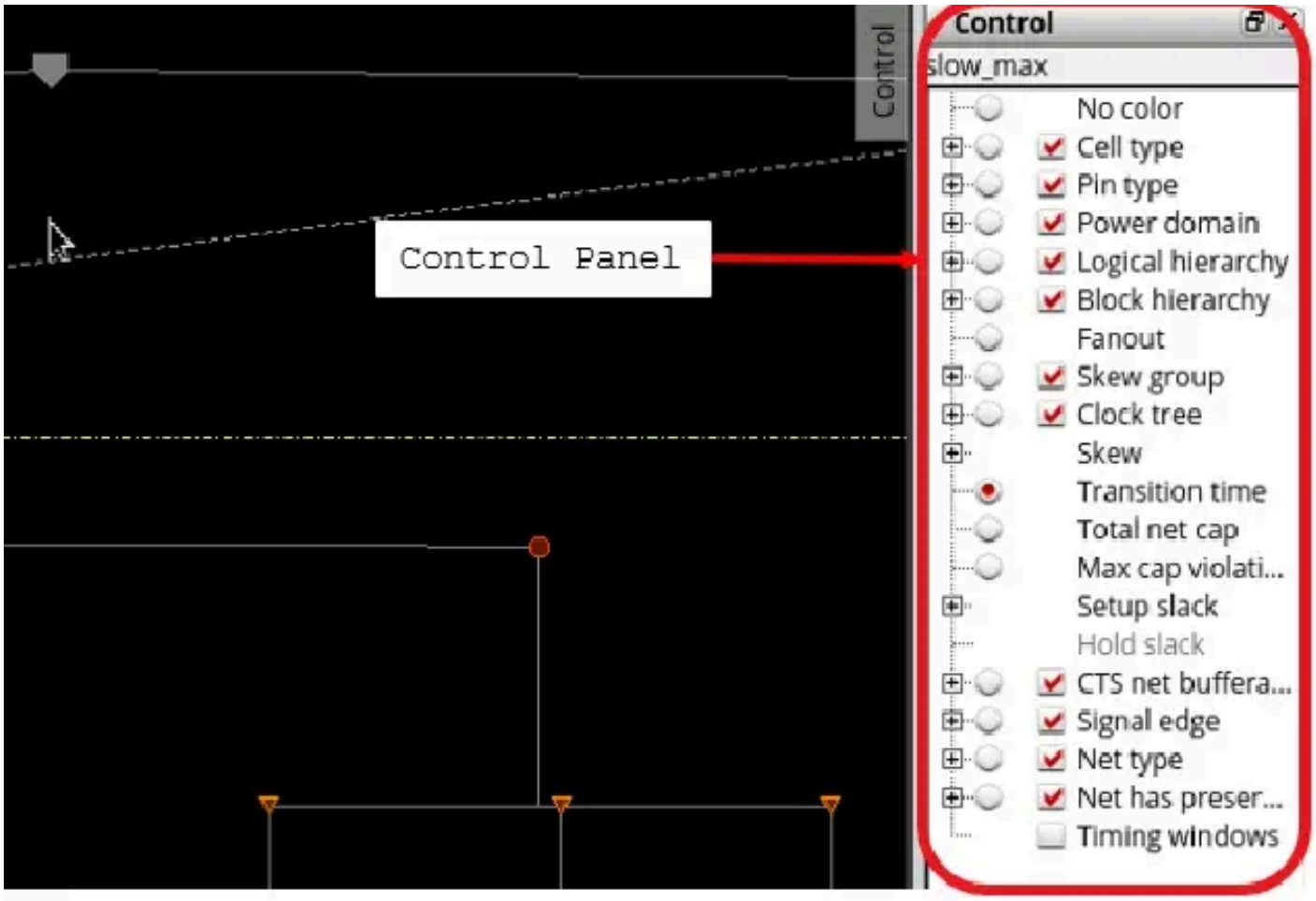


Tip: Cross-probing can be enabled/disabled using **View > Select > Enable crossprobing**.

The **Control Panel** combines the functionality of the **Visibility** and **Color by** menus into a single form.

© 2017 Cadence Design Systems, Inc. All rights reserved worldwide.

7. Select **View > Control Panel** or click the **Control** tab in the upper-right to open the Control Panel.



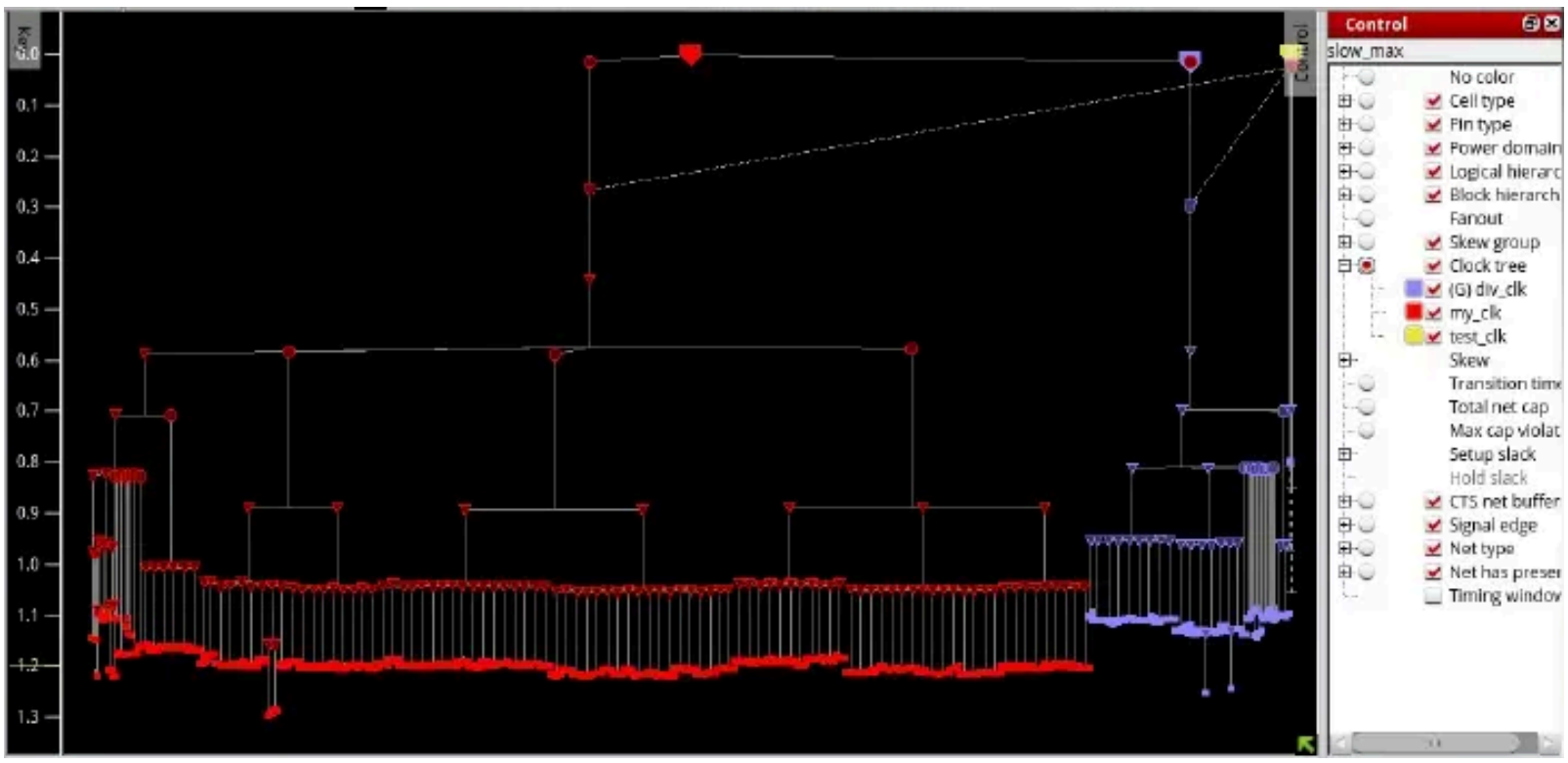
8. Expand the **Clock tree** object by clicking the '+' symbol next to it.



The radio buttons specify what the coloring is based on. You can change the color manually.

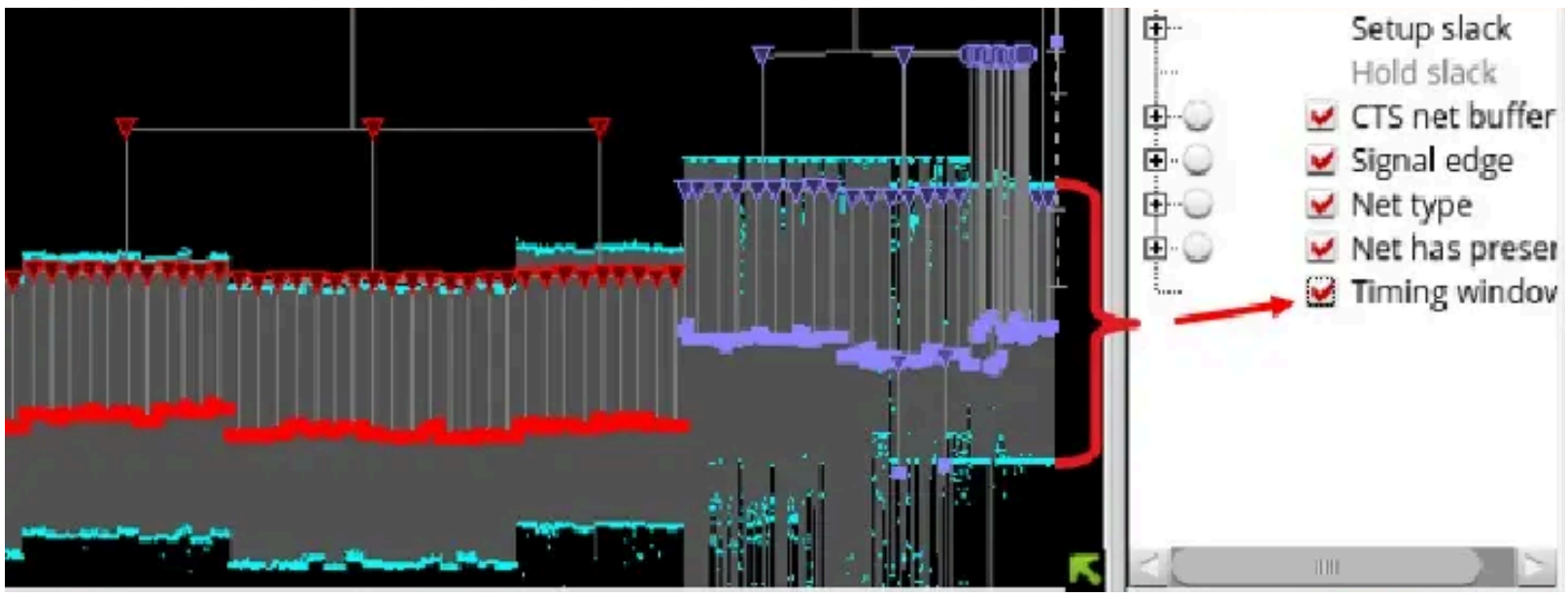
© 2017 Cadence Design Systems, Inc. All rights reserved worldwide.

9. Enable the **Clock tree** radio button. The clock tree will be colored with the default color.



Timing windows show the target delays that the CCOpt algorithm is aiming for as a range.

10. Enable the **Timing windows** check box in the Control Panel



Observe that the timing windows are shown for each sink.

11. Disable the **Timing windows** check box in the Control Panel.

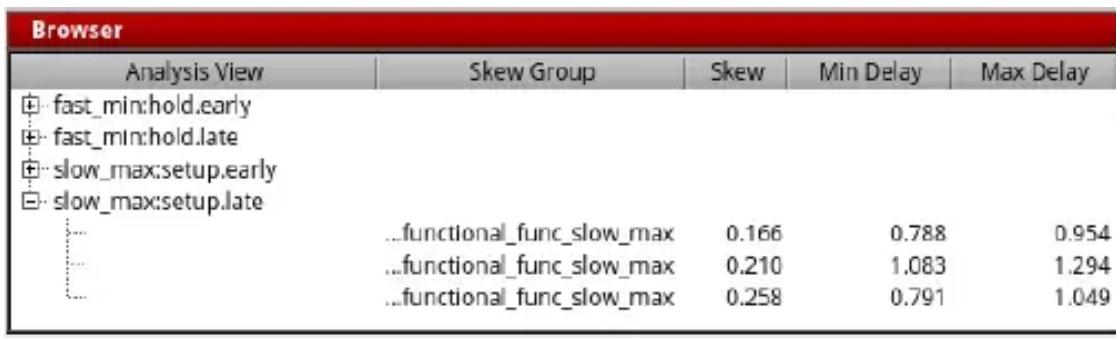
© 2017 Cadence Design Systems, Inc. All rights reserved worldwide.

12. Select **No Color** in the Control Panel in preparation for the next steps.



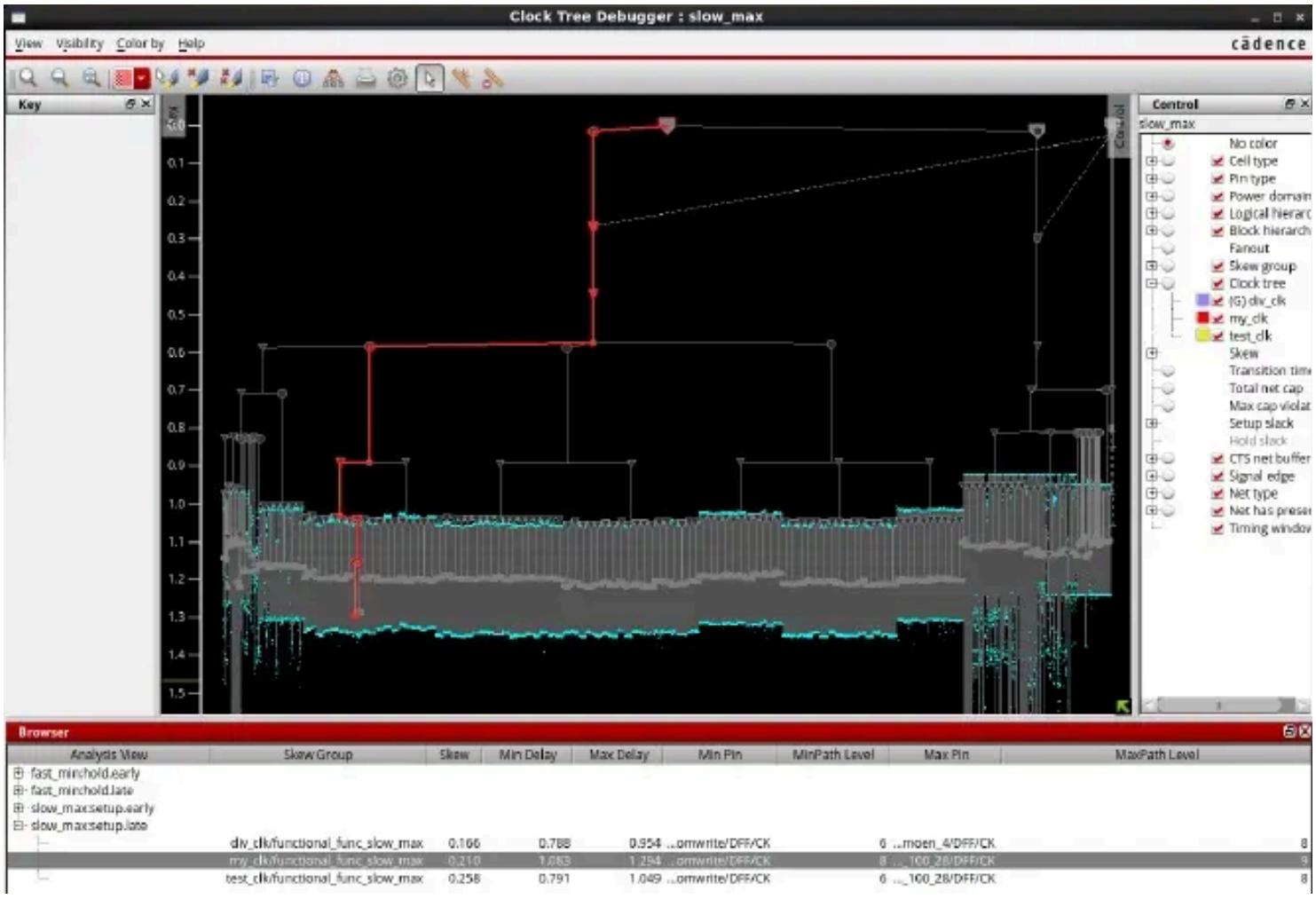
The Clock Path Browser displays the clock path data in a table and provides the option for bringing up a clock path analyzer. This is done either from its context menu or by double-clicking on a row in the table.

13. In the Browser, minimize the Analysis Views except for *slow_max:setup:late*, by clicking the '-' symbol.



14. Click the RMB on *my_clk/functional_func_slow_max* and select a color under **Highlight > Max Path**.

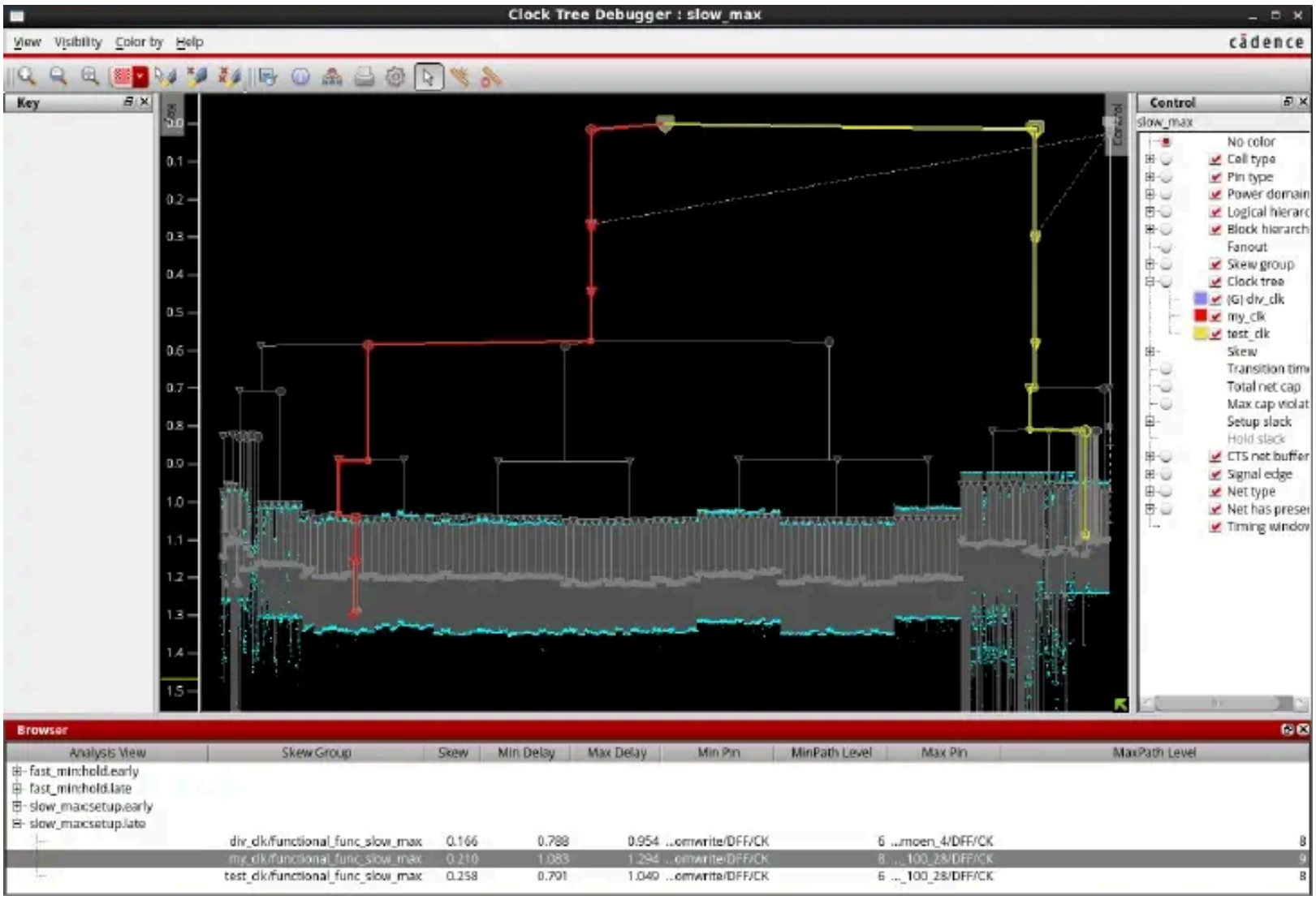
© 2017 Cadence Design Systems, Inc. All rights reserved worldwide.



Observe that the path with the longest delay for *my_clk* is highlighted.

© 2017 Cadence Design Systems, Inc. All rights reserved worldwide.

15. Highlight the minimum path in a similar way by clicking the RMB on *my_clk/functional_func_slow_max* and selecting a color under **Highlight > Min Path**.

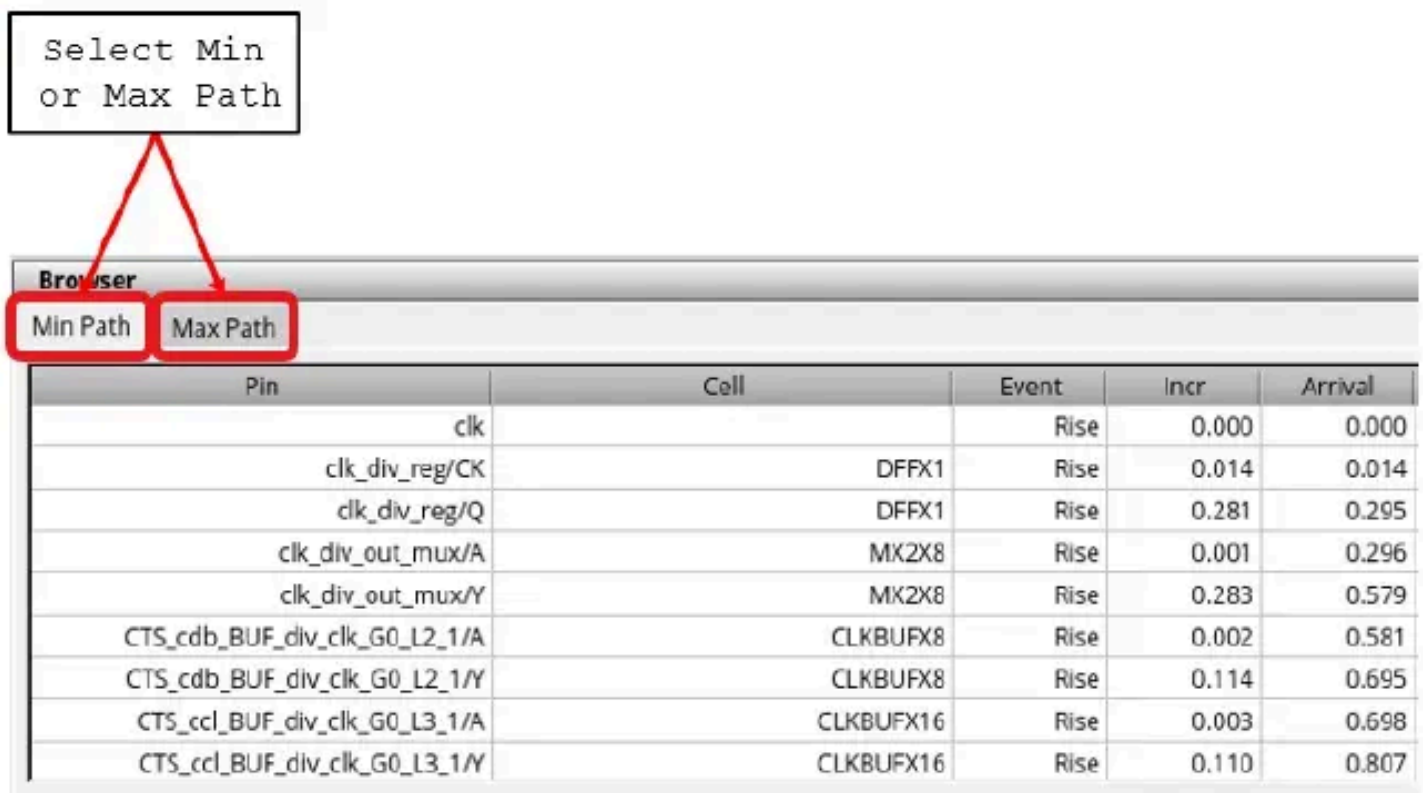


16. Clear the highlighting by clicking the RMB in the display area and selecting **Dehighlight All**.

The **Path Browser** enables you to see the detailed path delays for the min and max paths.

© 2017 Cadence Design Systems, Inc. All rights reserved worldwide.

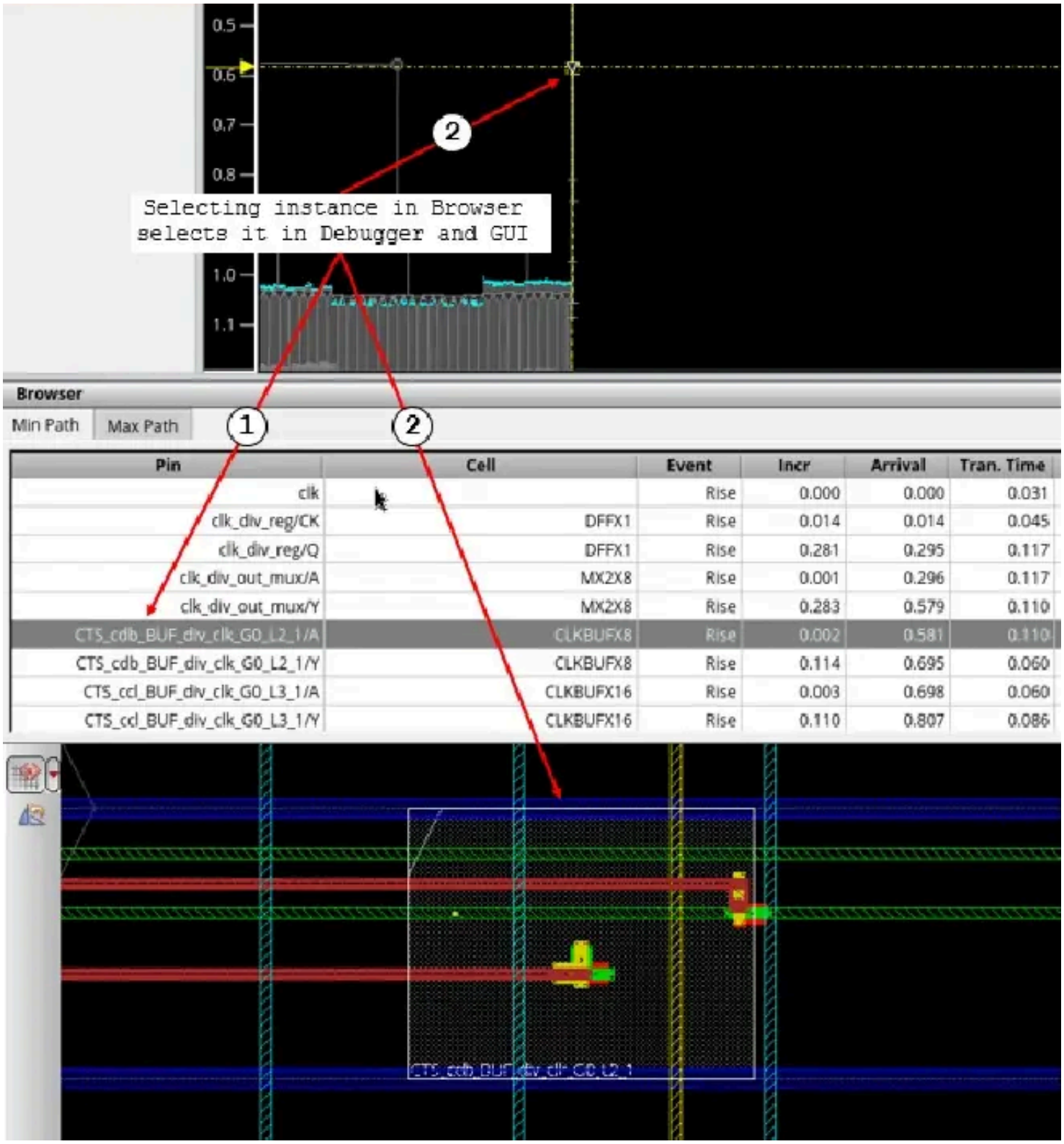
17. Double-click on *my_clk/functional_func_slow_max* in the Browser to drill down and view the details of the min and max paths.



Pin	Cell	Event	Incr	Arrival
clk		Rise	0.000	0.000
clk_div_reg/CK	DFFX1	Rise	0.014	0.014
clk_div_reg/Q	DFFX1	Rise	0.281	0.295
clk_div_out_mux/A	MX2X8	Rise	0.001	0.296
clk_div_out_mux/Y	MX2X8	Rise	0.283	0.579
CTS_cdb_BUF_div_clk_G0_L2_1/A	CLKBUFX8	Rise	0.002	0.581
CTS_cdb_BUF_div_clk_G0_L2_1/Y	CLKBUFX8	Rise	0.114	0.695
CTS_ccl_BUF_div_clk_G0_L3_1/A	CLKBUFX16	Rise	0.003	0.698
CTS_ccl_BUF_div_clk_G0_L3_1/Y	CLKBUFX16	Rise	0.110	0.807

© 2017 Cadence Design Systems, Inc. All rights reserved worldwide.

18. Select an instance in the path and observe that it is highlighted in the display.



© 2017 Cadence Design Systems, Inc. All rights reserved worldwide.

References

This concludes CCOpt Rapid Adoption Kit (RAK) for Beginners. The following are additional resources you can utilize to learn more about native CCOpt in the Innovus Implementation System:

• CCOpt Clock Tree Debugger in the Innovus Implementation System Menu Reference

• Clock Tree Synthesis in the Innovus Implementation System User Guide

• Clock Tree Synthesis Commands in the Innovus Implementation System Command Reference



© 2017 Cadence Design Systems, Inc. All rights reserved worldwide.